

Evaluating Model Agreement in Explainable Fake News Detection using Transformer Based Models

Using Transformer-Based Models (BERT, RoBERTa, DistilBERT)

Project by: Sheetal Gupta

Dataset: LIAR Benchmark | **XAI:** SHAP · LIME · BertViz Attention

1. Install Dependencies

```
!pip install transformers datasets accelerate shap lime bertviz optuna \
  textblob vaderSentiment textstat fairlearn xgboost \
  imbalanced-learn mlflow scipy pingouin -q
!python -m nltk.downloader stopwords -q
```

0.0/275.7 kB ? eta -:--:--
266.2/275.7 kB 40.4 MB/s eta
0:00:01 275.7/275.7 kB 9.4
MB/s eta 0:00:00
etadate (setup.py) ...
49.7/49.7 kB 2.5 MB/s eta 0:00:00
50.5/50.5 kB 4.2 MB/s eta
0:00:00
43.7/43.7 kB 1.8 MB/s eta
0:00:00
157.5/157.5 kB 10.5 MB/s eta
0:00:00
425.6/425.6 kB 34.6 MB/s eta
0:00:00
126.0/126.0 kB 10.6 MB/s eta
0:00:00
177.1/177.1 kB 10.1 MB/s eta
0:00:00
135.5/135.5 kB 12.1 MB/s eta
0:00:00
12.6/12.6 MB 70.1 MB/s eta
0:00:00
3.5/3.5 MB 72.8 MB/s eta
0:00:00
1.7/1.7 MB 64.2 MB/s eta
0:00:00
204.0/204.0 kB 15.5 MB/s eta

```

0:00:00
----- 147.8/147.8 kB 14.1 MB/s eta
0:00:00
----- 114.9/114.9 kB 12.2 MB/s eta
0:00:00
----- 212.0/212.0 kB 17.3 MB/s eta
0:00:00
----- 94.9/94.9 kB 9.7 MB/s eta
0:00:00
----- 132.2/132.2 kB 11.6 MB/s eta
0:00:00
----- 140.5/140.5 kB 6.6 MB/s eta
0:00:00
----- 2.1/2.1 MB 62.1 MB/s eta
0:00:00
----- 15.3/15.3 MB 77.4 MB/s eta
0:00:00
----- 939.7/939.7 kB 53.0 MB/s eta
0:00:00
----- 214.9/214.9 kB 16.9 MB/s eta
0:00:00
----- 4.9/4.9 MB 99.4 MB/s eta
0:00:00
----- 90.1/90.1 kB 5.5 MB/s eta
0:00:00
e (setup.py) ... eWarning: 'nltk.downloader' found in sys.modules
after import of package 'nltk', but prior to execution of
'nltk.downloader'; this may result in unpredictable behaviour

```

2. Imports

```

import pandas as pd
import numpy as np
import re, os, json, datetime, warnings
warnings.filterwarnings("ignore")

# NLP
import nltk
from nltk.corpus import stopwords
from textblob import TextBlob
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
import textstat

# ML
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    classification_report, confusion_matrix,
    roc_auc_score, roc_curve, cohen_kappa_score
)

```

```

)
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
import xgboost as xgb
from imblearn.over_sampling import SMOTE

# Deep Learning
import torch
from torch.utils.data import Dataset as TorchDataset
from transformers import (
    BertTokenizerFast, BertForSequenceClassification,
    RobertaTokenizerFast, RobertaForSequenceClassification,
    DistilBertTokenizerFast, DistilBertForSequenceClassification,
    Trainer, TrainingArguments,
    EarlyStoppingCallback
)

# XAI
import shap
from lime.lime_text import LimeTextExplainer
from bertviz import head_view

# Hyperparameter tuning
import optuna
optuna.logging.set_verbosity(optuna.logging.WARNING)

# Fairness
from fairlearn.metrics import demographic_parity_difference,
equalized_odds_difference

# Stats
from scipy import stats
import pingouin as pg

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Experiment tracking
import mlflow

print("All imports successful!")
print("GPU available:", torch.cuda.is_available())
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", DEVICE)

```

```
All imports successful!
GPU available: True
Using device: cuda
```

3. Load LIAR Dataset

```
columns = [
    "id", "label", "statement", "subject",
    "speaker", "speaker_job", "state", "party",
    "barely_true", "false_counts", "half_true",
    "mostly_true", "pants_fire", "context"
]

train_df = pd.read_csv("/content/train.tsv", sep='\t', header=None,
names=columns)
test_df = pd.read_csv("/content/test.tsv", sep='\t', header=None,
names=columns)
valid_df = pd.read_csv("/content/valid.tsv", sep='\t', header=None,
names=columns)

print(f"Train: {len(train_df)} | Valid: {len(valid_df)} | Test:
{len(test_df)}")
print("Label distribution (train):")
print(train_df['label'].value_counts())

Train: 10240 | Valid: 1284 | Test: 1267
Label distribution (train):
label
half-true      2114
false          1995
mostly-true    1962
true           1676
barely-true    1654
pants-fire      839
Name: count, dtype: int64
```

4. Label Encoding

Fake = 1 (false, pants-fire, barely-true) | Real = 0 (true, mostly-true, half-true)

```
# NOTE: fake=1, real=0 (consistent throughout)
fake_labels = ['false', 'pants-fire', 'barely-true']

def convert_label(label):
    return 1 if label in fake_labels else 0

for df in [train_df, test_df, valid_df]:
    df['target'] = df['label'].apply(convert_label)
```

```
print("Binary label distribution (train):")
print(train_df['target'].value_counts())
print("\nClass balance (train):", round(train_df['target'].mean(), 3))
```

```
Binary label distribution (train):
target
0      5752
1      4488
Name: count, dtype: int64
```

```
Class balance (train): 0.438
```

5. Text Cleaning & Feature Engineering

```
stop_words = set(stopwords.words('english'))
```

```
def clean_text(text):
    text = str(text).lower()
    text = re.sub(r"http\S+", "", text)
    text = re.sub(r"[^a-zA-Z ]", "", text)
    words = [w for w in text.split() if w not in stop_words]
    return " ".join(words)
```

```
for df in [train_df, test_df, valid_df]:
    df['clean_text'] = df['statement'].apply(clean_text)
```

```
print("Sample cleaned text:")
print(train_df['clean_text'].iloc[0])
```

```
Sample cleaned text:
says annies list political group supports thirdtrimester abortions
demand
```

5.1 Sentiment & Readability Features

```
analyzer = SentimentIntensityAnalyzer()
```

```
def sentiment_features(text):
    blob = TextBlob(text)
    vader = analyzer.polarity_scores(text)
    return pd.Series([blob.sentiment.polarity,
                      blob.sentiment.subjectivity, vader['compound']])
```

```
def readability_features(text):
    return pd.Series([
        textstat.flesch_reading_ease(text),
        textstat.lexicon_count(text),
        textstat.sentence_count(text)
    ])
```

```

for df in [train_df, test_df, valid_df]:
    df[['polarity', 'subjectivity', 'compound']] =
df['clean_text'].apply(sentiment_features)
    df[['readability', 'word_count', 'sentence_count']] =
df['clean_text'].apply(readability_features)

print("Feature columns added:",
      ['polarity', 'subjectivity', 'compound', 'readability', 'word_count', 'sentence_count'])
train_df[['polarity', 'subjectivity', 'compound', 'readability', 'word_count', 'sentence_count']].describe()

```

```

Feature columns added: ['polarity', 'subjectivity', 'compound',
                        'readability', 'word_count', 'sentence_count']

```

```

{"summary": "{\n  \"name\":\n  \"train_df[['polarity', 'subjectivity', 'compound', 'readability', 'word_count', 'sentence_count']]\",\n  \"rows\": 8,\n  \"fields\": {\n    \"column\": \"polarity\", \n    \"properties\": {\n      \"dtype\":\n      \"number\", \n      \"std\": 3620.3724858744345, \n      \"min\": -1.0, \n      \"max\": 10240.0, \n      \"num_unique_values\": 7, \n      \"samples\": [\n        10240.0, \n        0.022883902207547227, \n        0.06195887445887445\n      ], \n      \"semantic_type\": \"\", \n      \"description\": \"\"\n    }, \n    \"column\": \"subjectivity\", \n    \"properties\": {\n      \"dtype\":\n      \"number\", \n      \"std\": 3620.2866954181445, \n      \"min\": 0.0, \n      \"max\": 10240.0, \n      \"num_unique_values\": 7, \n      \"samples\": [\n        10240.0, \n        0.21898701480088198, \n        0.4\n      ], \n      \"semantic_type\": \"\", \n      \"description\": \"\"\n    }, \n    \"column\": \"compound\", \n    \"properties\": {\n      \"dtype\":\n      \"number\", \n      \"std\": 3620.3685920777834, \n      \"min\": -0.9744, \n      \"max\": 10240.0, \n      \"num_unique_values\": 8, \n      \"samples\": [\n        -0.005896787109375, \n        0.0, \n        10240.0\n      ], \n      \"semantic_type\": \"\", \n      \"description\": \"\"\n    }, \n    \"column\": \"readability\", \n    \"properties\": {\n      \"dtype\":\n      \"number\", \n      \"std\": 3624.205994133854, \n      \"min\": -301.78, \n      \"max\": 10240.0, \n      \"num_unique_values\": 8, \n      \"samples\": [\n        28.233666511114546, \n        30.530000000000003, \n        10240.0\n      ], \n      \"semantic_type\": \"\", \n      \"description\": \"\"\n    }, \n    \"column\": \"word_count\", \n    \"properties\": {\n      \"dtype\":\n      \"number\", \n      \"std\": 3609.2806548658705, \n      \"min\": 1.0, \n      \"max\": 10240.0, \n      \"num_unique_values\": 8, \n      \"samples\": [\n        10.51376953125, \n        10.0, \n        10240.0\n      ], \n      \"semantic_type\": \"\", \n      \"description\": \"\"\n    }, \n    \"column\": \"sentence_count\", \n    \"properties\": {\n      \"dtype\":\n      \"number\", \n      \"std\":

```

```
3620.083690824209,\n          \"min\": 0.0,\n          \"max\": 10240.0,\n          \"num_unique_values\": 3,\n          \"samples\": [\n              1.0,\n              0.0\n          ],\n          \"semantic_type\":\n          \"\", \n          \"description\": \"\"\n      }\n  ]\n}","type":"dataframe"}
```

6. TF-IDF Vectorization + SMOTE (Classical ML)

```
tfidf = TfidfVectorizer(max_features=5000)
X_train_tfidf = tfidf.fit_transform(train_df['clean_text'])
X_valid_tfidf = tfidf.transform(valid_df['clean_text'])
X_test_tfidf = tfidf.transform(test_df['clean_text'])

y_train = train_df['target']
y_valid = valid_df['target']
y_test = test_df['target']

smote = SMOTE(random_state=42)
X_train_smote, y_train_smote = smote.fit_resample(X_train_tfidf,
y_train)

print("Before SMOTE:", dict(pd.Series(y_train).value_counts()))
print("After SMOTE: ", dict(pd.Series(y_train_smote).value_counts()))

Before SMOTE: {0: np.int64(5752), 1: np.int64(4488)}
After SMOTE:  {1: np.int64(5752), 0: np.int64(5752)}
```

7. Classical ML Baselines (LR, RF, SVM, XGBoost)

```
classical_results = {}

# Logistic Regression
lr_model = LogisticRegression(max_iter=1000)
lr_model.fit(X_train_smote, y_train_smote)
lr_preds = lr_model.predict(X_test_tfidf)
lr_probs = lr_model.predict_proba(X_test_tfidf)[: , 1]
classical_results['Logistic Regression'] = {
    'accuracy': accuracy_score(y_test, lr_preds),
    'macro_f1': f1_score(y_test, lr_preds, average='macro'),
    'auc_roc': roc_auc_score(y_test, lr_probs),
    'preds': lr_preds,
    'probs': lr_probs
}

print("=== Logistic Regression ===")
print(classification_report(y_test, lr_preds,
target_names=['Real', 'Fake']))

=== Logistic Regression ===
           precision    recall  f1-score   support


```

Real	0.65	0.65	0.65	714
Fake	0.55	0.56	0.55	553
accuracy			0.61	1267
macro avg	0.60	0.60	0.60	1267
weighted avg	0.61	0.61	0.61	1267

Random Forest

```
rf_model = RandomForestClassifier(n_estimators=200, random_state=42)
rf_model.fit(X_train_smote, y_train_smote)
rf_preds = rf_model.predict(X_test_tfidf)
rf_probs = rf_model.predict_proba(X_test_tfidf)[:, 1]
classical_results['Random Forest'] = {
    'accuracy': accuracy_score(y_test, rf_preds),
    'macro_f1': f1_score(y_test, rf_preds, average='macro'),
    'auc_roc': roc_auc_score(y_test, rf_probs),
    'preds': rf_preds,
    'probs': rf_probs
}
print("=== Random Forest ===")
print(classification_report(y_test, rf_preds,
target_names=['Real', 'Fake']))
```

=== Random Forest ===

	precision	recall	f1-score	support
Real	0.63	0.65	0.64	714
Fake	0.53	0.51	0.52	553
accuracy			0.59	1267
macro avg	0.58	0.58	0.58	1267
weighted avg	0.59	0.59	0.59	1267

SVM

```
svm_model = SVC(probability=True, random_state=42)
svm_model.fit(X_train_smote, y_train_smote)
svm_preds = svm_model.predict(X_test_tfidf)
svm_probs = svm_model.predict_proba(X_test_tfidf)[:, 1]
classical_results['SVM'] = {
    'accuracy': accuracy_score(y_test, svm_preds),
    'macro_f1': f1_score(y_test, svm_preds, average='macro'),
    'auc_roc': roc_auc_score(y_test, svm_probs),
    'preds': svm_preds,
    'probs': svm_probs
}
print("=== SVM ===")
print(classification_report(y_test, svm_preds,
target_names=['Real', 'Fake']))
```


=== SVM ===

	precision	recall	f1-score	support
Real	0.64	0.77	0.70	714
Fake	0.59	0.43	0.50	553
accuracy			0.62	1267
macro avg	0.61	0.60	0.60	1267
weighted avg	0.62	0.62	0.61	1267

XGBoost

```
xgb_model = xgb.XGBClassifier(use_label_encoder=False,
                               eval_metric='logloss', random_state=42)
xgb_model.fit(X_train_smote, y_train_smote)
xgb_preds = xgb_model.predict(X_test_tfidf)
xgb_probs = xgb_model.predict_proba(X_test_tfidf)[:, 1]
classical_results['XGBoost'] = {
    'accuracy': accuracy_score(y_test, xgb_preds),
    'macro_f1': f1_score(y_test, xgb_preds, average='macro'),
    'auc_roc': roc_auc_score(y_test, xgb_probs),
    'preds': xgb_preds,
    'probs': xgb_probs
}
print("=== XGBoost ===")
print(classification_report(y_test, xgb_preds,
                             target_names=['Real', 'Fake']))
```

=== XGBoost ===

	precision	recall	f1-score	support
Real	0.62	0.70	0.66	714
Fake	0.54	0.45	0.49	553
accuracy			0.59	1267
macro avg	0.58	0.58	0.58	1267
weighted avg	0.59	0.59	0.59	1267

8. Transformer Fine-Tuning

8.1 PyTorch Dataset Class

```
class LIARDataset(TorchDataset):
    """PyTorch Dataset for LIAR statements."""
    def __init__(self, texts, labels, tokenizer, max_len=128):
        self.texts = texts
        self.labels = labels
        self.tokenizer = tokenizer
        self.max_len = max_len
```

```

def __len__(self):
    return len(self.texts)

def __getitem__(self, idx):
    encoding = self.tokenizer(
        self.texts[idx],
        max_length=self.max_len,
        padding='max_length',
        truncation=True,
        return_tensors='pt'
    )
    return {
        'input_ids': encoding['input_ids'].squeeze(),
        'attention_mask': encoding['attention_mask'].squeeze(),
        'labels': torch.tensor(self.labels[idx],
dtype=torch.long)
    }

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=1)
    acc = accuracy_score(labels, preds)
    macro_f1 = f1_score(labels, preds, average='macro')
    probs = torch.softmax(torch.tensor(logits), dim=1).numpy()[:, 1]
    auc = roc_auc_score(labels, probs)
    return {'accuracy': acc, 'macro_f1': macro_f1, 'auc_roc': auc}

print("Dataset class and compute_metrics defined.")

```

Dataset class and compute_metrics defined.

8.2 Optuna Hyperparameter Search (DistilBERT pilot)

```

# Run Optuna on DistilBERT to find best lr and batch_size before full training

distil_tokenizer =
DistilBertTokenizerFast.from_pretrained('distilbert-base-uncased')

# Use a small subset for fast Optuna search
OPTUNA_SUBSET = 1000
opt_texts = train_df['statement'].tolist()[:OPTUNA_SUBSET]
opt_labels = train_df['target'].tolist()[:OPTUNA_SUBSET]
opt_val_texts = valid_df['statement'].tolist()[:200]
opt_val_labels = valid_df['target'].tolist()[:200]

def optuna_objective(trial):
    lr = trial.suggest_float('learning_rate', 1e-5, 5e-5,
log=True)

```

```

batch_size = trial.suggest_categorical('batch_size', [8, 16])
epochs      = trial.suggest_int('epochs', 2, 3)

model = DistilBertForSequenceClassification.from_pretrained(
    'distilbert-base-uncased', num_labels=2
)

train_ds = LIARDataset(opt_texts,      opt_labels,
distil_tokenizer)
val_ds   = LIARDataset(opt_val_texts, opt_val_labels,
distil_tokenizer)

args = TrainingArguments(
    output_dir='./optuna_trial',
    num_train_epochs=epochs,
    per_device_train_batch_size=batch_size,
    per_device_eval_batch_size=32,
    learning_rate=lr,
    eval_strategy='epoch',
    save_strategy='no',
    logging_steps=50,
    report_to='none',
    disable_tqdm=True
)

trainer = Trainer(
    model=model,
    args=args,
    train_dataset=train_ds,
    eval_dataset=val_ds,
    compute_metrics=compute_metrics
)
trainer.train()
results = trainer.evaluate()
return results['eval_accuracy']

study = optuna.create_study(direction='maximize')
study.optimize(optuna_objective, n_trials=10) # 10 trials; increase
to 50 on full GPU run

BEST_LR      = study.best_params['learning_rate']
BEST_BATCH_SIZE = study.best_params['batch_size']
BEST_EPOCHS   = study.best_params['epochs']

print("\nBest hyperparameters found:")
print(f"  Learning rate : {BEST_LR}")
print(f"  Batch size      : {BEST_BATCH_SIZE}")
print(f"  Epochs          : {BEST_EPOCHS}")

```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

```
{"model_id": "9036718769cd4f4d8bc032758a50f78d", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "1a075adc73b04e95945d048c965da2c1", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "87a8275072264810ac18cb022031417a", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "3ee61702040e4c8992891bc476473821", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "598d4495ccdb459db90b660d96bbd3f0", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "c22bb4f788bd4f95a1fdb836582462c0", "version_major": 2, "version_minor": 0}
```

[transformers] DistilBertForSequenceClassification LOAD REPORT from: distilbert-base-uncased

Key	Status
-----+-----	-----+-----
vocab_layer_norm.bias	UNEXPECTED
vocab_projector.bias	UNEXPECTED
vocab_layer_norm.weight	UNEXPECTED
vocab_transform.weight	UNEXPECTED
vocab_transform.bias	UNEXPECTED
pre_classifier.weight	MISSING
classifier.bias	MISSING
classifier.weight	MISSING
pre_classifier.bias	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

```
{'loss': '0.6764', 'grad_norm': '1.488', 'learning_rate': '1.726e-05', 'epoch': '0.7937'}
```

```
{'eval_loss': '0.6688', 'eval_accuracy': '0.6', 'eval_macro_f1': '0.5931', 'eval_auc_roc': '0.6554', 'eval_runtime': '0.6748', 'eval_samples_per_second': '296.4', 'eval_steps_per_second': '10.37', 'epoch': '1'}
```

```
{'loss': '0.6382', 'grad_norm': '2.68', 'learning_rate': '1.11e-05',
'epoch': '1.587'}
{'eval_loss': '0.6654', 'eval_accuracy': '0.63', 'eval_macro_f1':
'0.63', 'eval_auc_roc': '0.6546', 'eval_runtime': '0.6731',
'eval_samples_per_second': '297.1', 'eval_steps_per_second': '10.4',
'epoch': '2'}
{'loss': '0.5825', 'grad_norm': '2.832', 'learning_rate': '4.931e-06',
'epoch': '2.381'}
{'eval_loss': '0.6754', 'eval_accuracy': '0.61', 'eval_macro_f1':
'0.6086', 'eval_auc_roc': '0.656', 'eval_runtime': '0.653',
'eval_samples_per_second': '306.3', 'eval_steps_per_second': '10.72',
'epoch': '3'}
{'train_runtime': '32.22', 'train_samples_per_second': '93.11',
'train_steps_per_second': '5.866', 'train_loss': '0.6156', 'epoch':
'3'}
{'eval_loss': '0.6754', 'eval_accuracy': '0.61', 'eval_macro_f1':
'0.6086', 'eval_auc_roc': '0.656', 'eval_runtime': '0.6601',
'eval_samples_per_second': '303', 'eval_steps_per_second': '10.6',
'epoch': '3'}
```

```
{"model_id": "dce9627221634a0190445d9b0686df98", "version_major": 2, "version_minor": 0}
```

[transformers] DistilBertForSequenceClassification LOAD REPORT from:
distilbert-base-uncased

Key	Status
-----+-----	-----+-----
vocab_layer_norm.bias	UNEXPECTED
vocab_projector.bias	UNEXPECTED
vocab_layer_norm.weight	UNEXPECTED
vocab_transform.weight	UNEXPECTED
vocab_transform.bias	UNEXPECTED
pre_classifier.weight	MISSING
classifier.bias	MISSING
classifier.weight	MISSING
pre_classifier.bias	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

```
{'loss': '0.6817', 'grad_norm': '2.546', 'learning_rate': '1.167e-05',
'epoch': '0.4'}
{'loss': '0.67', 'grad_norm': '2.007', 'learning_rate': '9.882e-06',
'epoch': '0.8'}
{'eval_loss': '0.6774', 'eval_accuracy': '0.59', 'eval_macro_f1':
'0.525', 'eval_auc_roc': '0.671', 'eval_runtime': '0.6925',
'eval_samples_per_second': '288.8', 'eval_steps_per_second': '10.11',
```

```

'epoch': '1'}
{'loss': '0.6586', 'grad_norm': '4.82', 'learning_rate': '8.091e-06',
'epoch': '1.2'}
{'loss': '0.6241', 'grad_norm': '2.802', 'learning_rate': '6.301e-06',
'epoch': '1.6'}
{'loss': '0.6165', 'grad_norm': '4.076', 'learning_rate': '4.511e-06',
'epoch': '2'}
{'eval_loss': '0.6594', 'eval_accuracy': '0.625', 'eval_macro_f1':
'0.625', 'eval_auc_roc': '0.6705', 'eval_runtime': '0.7034',
'eval_samples_per_second': '284.3', 'eval_steps_per_second': '9.951',
'epoch': '2'}
{'loss': '0.578', 'grad_norm': '5.12', 'learning_rate': '2.721e-06',
'epoch': '2.4'}
{'loss': '0.5892', 'grad_norm': '3.218', 'learning_rate': '9.309e-07',
'epoch': '2.8'}
{'eval_loss': '0.6651', 'eval_accuracy': '0.6', 'eval_macro_f1':
'0.6', 'eval_auc_roc': '0.677', 'eval_runtime': '0.7341',
'eval_samples_per_second': '272.4', 'eval_steps_per_second': '9.535',
'epoch': '3'}
{'train_runtime': '35.81', 'train_samples_per_second': '83.78',
'train_steps_per_second': '10.47', 'train_loss': '0.625', 'epoch':
'3'}
{'eval_loss': '0.6651', 'eval_accuracy': '0.6', 'eval_macro_f1':
'0.6', 'eval_auc_roc': '0.677', 'eval_runtime': '0.7906',
'eval_samples_per_second': '253', 'eval_steps_per_second': '8.854',
'epoch': '3'}

```

```

{"model_id": "17e1929743f54644ac5bf9cc7bb07", "version_major": 2, "version_minor": 0}

```

[transformers] DistilBertForSequenceClassification LOAD REPORT from:
distilbert-base-uncased

Key	Status
-----+-----	-----+-----
vocab_layer_norm.bias	UNEXPECTED
vocab_projector.bias	UNEXPECTED
vocab_layer_norm.weight	UNEXPECTED
vocab_transform.weight	UNEXPECTED
vocab_transform.bias	UNEXPECTED
pre_classifier.weight	MISSING
classifier.bias	MISSING
classifier.weight	MISSING
pre_classifier.bias	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

```
{'loss': '0.685', 'grad_norm': '2.313', 'learning_rate': '2.234e-05',
'epoch': '0.4'}
{'loss': '0.6691', 'grad_norm': '1.78', 'learning_rate': '1.678e-05',
'epoch': '0.8'}
{'eval_loss': '0.6799', 'eval_accuracy': '0.605', 'eval_macro_f1':
'0.5584', 'eval_auc_roc': '0.6911', 'eval_runtime': '0.7442',
'eval_samples_per_second': '268.7', 'eval_steps_per_second': '9.406',
'epoch': '1'}
{'loss': '0.6532', 'grad_norm': '5.336', 'learning_rate': '1.123e-05',
'epoch': '1.2'}
{'loss': '0.6035', 'grad_norm': '2.923', 'learning_rate': '5.669e-06',
'epoch': '1.6'}
{'loss': '0.5933', 'grad_norm': '4.082', 'learning_rate': '1.112e-07',
'epoch': '2'}
{'eval_loss': '0.6624', 'eval_accuracy': '0.62', 'eval_macro_f1':
'0.6197', 'eval_auc_roc': '0.6895', 'eval_runtime': '0.7588',
'eval_samples_per_second': '263.6', 'eval_steps_per_second': '9.225',
'epoch': '2'}
{'train_runtime': '25.77', 'train_samples_per_second': '77.6',
'train_steps_per_second': '9.7', 'train_loss': '0.6408', 'epoch': '2'}
{'eval_loss': '0.6624', 'eval_accuracy': '0.62', 'eval_macro_f1':
'0.6197', 'eval_auc_roc': '0.6895', 'eval_runtime': '0.7502',
'eval_samples_per_second': '266.6', 'eval_steps_per_second': '9.331',
'epoch': '2'}
```

```
{"model_id": "fb69a541ec054ef4b967aab042b3bfca", "version_major": 2, "version_minor": 0}
```

[transformers] DistilBertForSequenceClassification LOAD REPORT from:
distilbert-base-uncased

Key	Status
-----+-----+	
vocab_layer_norm.bias	UNEXPECTED
vocab_projector.bias	UNEXPECTED
vocab_layer_norm.weight	UNEXPECTED
vocab_transform.weight	UNEXPECTED
vocab_transform.bias	UNEXPECTED
pre_classifier.weight	MISSING
classifier.bias	MISSING
classifier.weight	MISSING
pre_classifier.bias	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

```
{'loss': '0.6879', 'grad_norm': '2.246', 'learning_rate': '2.234e-05',
'epoch': '0.4'}
```

```
{'loss': '0.6717', 'grad_norm': '1.671', 'learning_rate': '1.678e-05',
'epoch': '0.8'}
{'eval_loss': '0.6838', 'eval_accuracy': '0.58', 'eval_macro_f1':
'0.5251', 'eval_auc_roc': '0.6782', 'eval_runtime': '0.7812',
'eval_samples_per_second': '256', 'eval_steps_per_second': '8.96',
'epoch': '1'}
{'loss': '0.6537', 'grad_norm': '4.296', 'learning_rate': '1.122e-05',
'epoch': '1.2'}
{'loss': '0.6036', 'grad_norm': '3.208', 'learning_rate': '5.667e-06',
'epoch': '1.6'}
{'loss': '0.5915', 'grad_norm': '3.47', 'learning_rate': '1.111e-07',
'epoch': '2'}
{'eval_loss': '0.664', 'eval_accuracy': '0.655', 'eval_macro_f1':
'0.6546', 'eval_auc_roc': '0.6849', 'eval_runtime': '0.8061',
'eval_samples_per_second': '248.1', 'eval_steps_per_second': '8.684',
'epoch': '2'}
{'train_runtime': '26.37', 'train_samples_per_second': '75.85',
'train_steps_per_second': '9.481', 'train_loss': '0.6417', 'epoch':
'2'}
{'eval_loss': '0.664', 'eval_accuracy': '0.655', 'eval_macro_f1':
'0.6546', 'eval_auc_roc': '0.6849', 'eval_runtime': '0.8082',
'eval_samples_per_second': '247.5', 'eval_steps_per_second': '8.662',
'epoch': '2'}
```

```
{"model_id": "5b868d2073b24e10ae494f7e02e5d9fd", "version_major": 2, "version_minor": 0}
```

[transformers] DistilBertForSequenceClassification LOAD REPORT from:
distilbert-base-uncased

Key	Status
-----+-----+	
vocab_layer_norm.bias	UNEXPECTED
vocab_projector.bias	UNEXPECTED
vocab_layer_norm.weight	UNEXPECTED
vocab_transform.weight	UNEXPECTED
vocab_transform.bias	UNEXPECTED
pre_classifier.weight	MISSING
classifier.bias	MISSING
classifier.weight	MISSING
pre_classifier.bias	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

```
{'loss': '0.6873', 'grad_norm': '2.454', 'learning_rate': '1.541e-05',
'epoch': '0.4'}
{'loss': '0.671', 'grad_norm': '1.929', 'learning_rate': '1.158e-05',
```



```

'epoch': '0.8'}
{'eval_loss': '0.6733', 'eval_accuracy': '0.59', 'eval_macro_f1':
'0.5432', 'eval_auc_roc': '0.6778', 'eval_runtime': '0.8409',
'eval_samples_per_second': '237.9', 'eval_steps_per_second': '8.325',
'epoch': '1'}
{'loss': '0.6548', 'grad_norm': '4.944', 'learning_rate': '7.744e-06',
'epoch': '1.2'}
{'loss': '0.614', 'grad_norm': '3.061', 'learning_rate': '3.91e-06',
'epoch': '1.6'}
{'loss': '0.6085', 'grad_norm': '3.672', 'learning_rate': '7.667e-08',
'epoch': '2'}
{'eval_loss': '0.6579', 'eval_accuracy': '0.615', 'eval_macro_f1':
'0.6122', 'eval_auc_roc': '0.6834', 'eval_runtime': '0.8474',
'eval_samples_per_second': '236', 'eval_steps_per_second': '8.261',
'epoch': '2'}
{'train_runtime': '27.83', 'train_samples_per_second': '71.88',
'train_steps_per_second': '8.984', 'train_loss': '0.6471', 'epoch':
'2'}
{'eval_loss': '0.6579', 'eval_accuracy': '0.615', 'eval_macro_f1':
'0.6122', 'eval_auc_roc': '0.6834', 'eval_runtime': '0.8466',
'eval_samples_per_second': '236.2', 'eval_steps_per_second': '8.268',
'epoch': '2'}

```

```

{"model_id": "71d9afeeba3c4a5d917256b99ef33e4e", "version_major": 2, "vers
ion_minor": 0}

```

[transformers] DistilBertForSequenceClassification LOAD REPORT from:
distilbert-base-uncased

Key	Status
-----+-----	-----+-----
vocab_layer_norm.bias	UNEXPECTED
vocab_projector.bias	UNEXPECTED
vocab_layer_norm.weight	UNEXPECTED
vocab_transform.weight	UNEXPECTED
vocab_transform.bias	UNEXPECTED
pre_classifier.weight	MISSING
classifier.bias	MISSING
classifier.weight	MISSING
pre_classifier.bias	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

```

{'loss': '0.687', 'grad_norm': '2.227', 'learning_rate': '2.825e-05',
'epoch': '0.4'}
{'loss': '0.6727', 'grad_norm': '1.432', 'learning_rate': '2.122e-05',
'epoch': '0.8'}

```

```
{'eval_loss': '0.6961', 'eval_accuracy': '0.525', 'eval_macro_f1':
'0.4203', 'eval_auc_roc': '0.67', 'eval_runtime': '0.7747',
'eval_samples_per_second': '258.2', 'eval_steps_per_second': '9.036',
'epoch': '1'}
{'loss': '0.6568', 'grad_norm': '4.011', 'learning_rate': '1.42e-05',
'epoch': '1.2'}
{'loss': '0.6', 'grad_norm': '3.151', 'learning_rate': '7.168e-06',
'epoch': '1.6'}
{'loss': '0.5856', 'grad_norm': '3.093', 'learning_rate': '1.406e-07',
'epoch': '2'}
{'eval_loss': '0.6714', 'eval_accuracy': '0.635', 'eval_macro_f1':
'0.6349', 'eval_auc_roc': '0.6781', 'eval_runtime': '0.7689',
'eval_samples_per_second': '260.1', 'eval_steps_per_second': '9.104',
'epoch': '2'}
{'train_runtime': '26.44', 'train_samples_per_second': '75.64',
'train_steps_per_second': '9.456', 'train_loss': '0.6404', 'epoch':
'2'}
{'eval_loss': '0.6714', 'eval_accuracy': '0.635', 'eval_macro_f1':
'0.6349', 'eval_auc_roc': '0.6781', 'eval_runtime': '0.77',
'eval_samples_per_second': '259.7', 'eval_steps_per_second': '9.091',
'epoch': '2'}
```

```
{"model_id": "289b59f77fa94ef79278f8f24a799ba3", "version_major": 2, "vers
ion_minor": 0}
```

[transformers] DistilBertForSequenceClassification LOAD REPORT from:
distilbert-base-uncased

Key	Status
vocab_layer_norm.bias	UNEXPECTED
vocab_projector.bias	UNEXPECTED
vocab_layer_norm.weight	UNEXPECTED
vocab_transform.weight	UNEXPECTED
vocab_transform.bias	UNEXPECTED
pre_classifier.weight	MISSING
classifier.bias	MISSING
classifier.weight	MISSING
pre_classifier.bias	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

```
{'loss': '0.6868', 'grad_norm': '2.32', 'learning_rate': '9.673e-06',
'epoch': '0.4'}
{'loss': '0.6738', 'grad_norm': '1.886', 'learning_rate': '8.189e-06',
'epoch': '0.8'}
{'eval_loss': '0.6787', 'eval_accuracy': '0.545', 'eval_macro_f1':
```

```

'0.4388', 'eval_auc_roc': '0.6661', 'eval_runtime': '0.783',
'eval_samples_per_second': '255.4', 'eval_steps_per_second': '8.94',
'epoch': '1'}
{'loss': '0.6624', 'grad_norm': '4.754', 'learning_rate': '6.706e-06',
'epoch': '1.2'}
{'loss': '0.6309', 'grad_norm': '3.073', 'learning_rate': '5.222e-06',
'epoch': '1.6'}
{'loss': '0.6294', 'grad_norm': '3.495', 'learning_rate': '3.739e-06',
'epoch': '2'}
{'eval_loss': '0.6596', 'eval_accuracy': '0.615', 'eval_macro_f1':
'0.6115', 'eval_auc_roc': '0.6727', 'eval_runtime': '0.7969',
'eval_samples_per_second': '251', 'eval_steps_per_second': '8.784',
'epoch': '2'}
{'loss': '0.5963', 'grad_norm': '3.375', 'learning_rate': '2.255e-06',
'epoch': '2.4'}
{'loss': '0.6151', 'grad_norm': '2.998', 'learning_rate': '7.715e-07',
'epoch': '2.8'}
{'eval_loss': '0.6578', 'eval_accuracy': '0.63', 'eval_macro_f1':
'0.6282', 'eval_auc_roc': '0.6764', 'eval_runtime': '0.8117',
'eval_samples_per_second': '246.4', 'eval_steps_per_second': '8.624',
'epoch': '3'}
{'train_runtime': '39.85', 'train_samples_per_second': '75.29',
'train_steps_per_second': '9.411', 'train_loss': '0.6371', 'epoch':
'3'}
{'eval_loss': '0.6578', 'eval_accuracy': '0.63', 'eval_macro_f1':
'0.6282', 'eval_auc_roc': '0.6764', 'eval_runtime': '0.8033',
'eval_samples_per_second': '249', 'eval_steps_per_second': '8.714',
'epoch': '3'}

```

```

{"model_id": "ca9090cf91f04a79900131d9538f1a7f", "version_major": 2, "vers
ion_minor": 0}

```

[transformers] DistilBertForSequenceClassification LOAD REPORT from:
distilbert-base-uncased

Key	Status
-----+-----	-----+-----
vocab_layer_norm.bias	UNEXPECTED
vocab_projector.bias	UNEXPECTED
vocab_layer_norm.weight	UNEXPECTED
vocab_transform.weight	UNEXPECTED
vocab_transform.bias	UNEXPECTED
pre_classifier.weight	MISSING
classifier.bias	MISSING
classifier.weight	MISSING
pre_classifier.bias	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.

- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

```
{'loss': '0.6822', 'grad_norm': '1.184', 'learning_rate': '6.906e-06',  
'epoch': '0.7937'}  
{'eval_loss': '0.6884', 'eval_accuracy': '0.51', 'eval_macro_f1':  
'0.3552', 'eval_auc_roc': '0.6382', 'eval_runtime': '0.8177',  
'eval_samples_per_second': '244.6', 'eval_steps_per_second': '8.56',  
'epoch': '1'}  
{'loss': '0.6593', 'grad_norm': '1.77', 'learning_rate': '2.422e-06',  
'epoch': '1.587'}  
{'eval_loss': '0.6852', 'eval_accuracy': '0.54', 'eval_macro_f1':  
'0.4165', 'eval_auc_roc': '0.6418', 'eval_runtime': '0.8057',  
'eval_samples_per_second': '248.2', 'eval_steps_per_second': '8.689',  
'epoch': '2'}  
{'train_runtime': '24.95', 'train_samples_per_second': '80.17',  
'train_steps_per_second': '5.051', 'train_loss': '0.6655', 'epoch':  
'2'}  
{'eval_loss': '0.6852', 'eval_accuracy': '0.54', 'eval_macro_f1':  
'0.4165', 'eval_auc_roc': '0.6418', 'eval_runtime': '0.8246',  
'eval_samples_per_second': '242.5', 'eval_steps_per_second': '8.489',  
'epoch': '2'}
```

```
{"model_id": "9994d09ad7b44569a36e5bc3db905f00", "version_major": 2, "version_minor": 0}
```

[transformers] DistilBertForSequenceClassification LOAD REPORT from:
distilbert-base-uncased

Key	Status
-----+-----	-----+-----
vocab_layer_norm.bias	UNEXPECTED
vocab_projector.bias	UNEXPECTED
vocab_layer_norm.weight	UNEXPECTED
vocab_transform.weight	UNEXPECTED
vocab_transform.bias	UNEXPECTED
pre_classifier.weight	MISSING
classifier.bias	MISSING
classifier.weight	MISSING
pre_classifier.bias	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

```
{'loss': '0.6787', 'grad_norm': '1.316', 'learning_rate': '7.434e-06',  
'epoch': '0.7937'}  
{'eval_loss': '0.6837', 'eval_accuracy': '0.52', 'eval_macro_f1':  
'0.3763', 'eval_auc_roc': '0.6625', 'eval_runtime': '0.803',
```

```

'eval_samples_per_second': '249.1', 'eval_steps_per_second': '8.717',
'epoch': '1'}
{'loss': '0.6592', 'grad_norm': '2.169', 'learning_rate': '4.779e-06',
'epoch': '1.587'}
{'eval_loss': '0.675', 'eval_accuracy': '0.57', 'eval_macro_f1':
'0.4974', 'eval_auc_roc': '0.6703', 'eval_runtime': '0.8028',
'eval_samples_per_second': '249.1', 'eval_steps_per_second': '8.719',
'epoch': '2'}
{'loss': '0.6405', 'grad_norm': '1.823', 'learning_rate': '2.124e-06',
'epoch': '2.381'}
{'eval_loss': '0.6654', 'eval_accuracy': '0.625', 'eval_macro_f1':
'0.6102', 'eval_auc_roc': '0.669', 'eval_runtime': '0.8013',
'eval_samples_per_second': '249.6', 'eval_steps_per_second': '8.736',
'epoch': '3'}
{'train_runtime': '36.94', 'train_samples_per_second': '81.22',
'train_steps_per_second': '5.117', 'train_loss': '0.6527', 'epoch':
'3'}
{'eval_loss': '0.6654', 'eval_accuracy': '0.625', 'eval_macro_f1':
'0.6102', 'eval_auc_roc': '0.669', 'eval_runtime': '0.8013',
'eval_samples_per_second': '249.6', 'eval_steps_per_second': '8.735',
'epoch': '3'}

```

```

{"model_id": "b9a2fbfb6ec04080b20ff6d386061e98", "version_major": 2, "vers
ion_minor": 0}

```

[transformers] DistilBertForSequenceClassification LOAD REPORT from:
distilbert-base-uncased

Key	Status
-----+-----	-----+-----
vocab_layer_norm.bias	UNEXPECTED
vocab_projector.bias	UNEXPECTED
vocab_layer_norm.weight	UNEXPECTED
vocab_transform.weight	UNEXPECTED
vocab_transform.bias	UNEXPECTED
pre_classifier.weight	MISSING
classifier.bias	MISSING
classifier.weight	MISSING
pre_classifier.bias	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

```

{'loss': '0.6874', 'grad_norm': '2.168', 'learning_rate': '2.996e-05',
'epoch': '0.4'}
{'loss': '0.6699', 'grad_norm': '1.836', 'learning_rate': '2.251e-05',
'epoch': '0.8'}
{'eval_loss': '0.6849', 'eval_accuracy': '0.6', 'eval_macro_f1':

```

```

'0.5575', 'eval_auc_roc': '0.6946', 'eval_runtime': '0.802',
'eval_samples_per_second': '249.4', 'eval_steps_per_second': '8.729',
'epoch': '1'}
{'loss': '0.6514', 'grad_norm': '4.916', 'learning_rate': '1.505e-05',
'epoch': '1.2'}
{'loss': '0.593', 'grad_norm': '3.029', 'learning_rate': '7.601e-06',
'epoch': '1.6'}
{'loss': '0.5783', 'grad_norm': '4.226', 'learning_rate': '1.49e-07',
'epoch': '2'}
{'eval_loss': '0.6616', 'eval_accuracy': '0.62', 'eval_macro_f1':
'0.6162', 'eval_auc_roc': '0.6987', 'eval_runtime': '0.8014',
'eval_samples_per_second': '249.6', 'eval_steps_per_second': '8.735',
'epoch': '2'}
{'train_runtime': '26.83', 'train_samples_per_second': '74.56',
'train_steps_per_second': '9.319', 'train_loss': '0.636', 'epoch':
'2'}
{'eval_loss': '0.6616', 'eval_accuracy': '0.62', 'eval_macro_f1':
'0.6162', 'eval_auc_roc': '0.6987', 'eval_runtime': '0.8061',
'eval_samples_per_second': '248.1', 'eval_steps_per_second': '8.683',
'epoch': '2'}

```

Best hyperparameters found:

Learning rate : 2.7781658345974626e-05

Batch size : 8

Epochs : 2

8.3 Fine-Tune BERT, RoBERTa, DistilBERT

```

transformer_configs = {
    'DistilBERT': {
        'model_name': 'distilbert-base-uncased',
        'tokenizer_cls': DistilBertTokenizerFast,
        'model_cls': DistilBertForSequenceClassification,
    },
    'BERT': {
        'model_name': 'bert-base-uncased',
        'tokenizer_cls': BertTokenizerFast,
        'model_cls': BertForSequenceClassification,
    },
    'RoBERTa': {
        'model_name': 'roberta-base',
        'tokenizer_cls': RobertaTokenizerFast,
        'model_cls': RobertaForSequenceClassification,
    },
}

transformer_results = {}
trained_models = {}
trained_tokenizers = {}

```

```

train_texts = train_df['statement'].tolist()
train_labels = train_df['target'].tolist()
valid_texts = valid_df['statement'].tolist()
valid_labels = valid_df['target'].tolist()
test_texts = test_df['statement'].tolist()
test_labels = test_df['target'].tolist()

mlflow.set_experiment("fake_news_transformers")

for model_name, cfg in transformer_configs.items():
    print(f"\n{'='*50}")
    print(f"Training {model_name}...")
    print(f"{'='*50}")

    tokenizer =
cfg['tokenizer_cls'].from_pretrained(cfg['model_name'])
    model = cfg['model_cls'].from_pretrained(cfg['model_name'],
num_labels=2)

    train_ds = LIARDataset(train_texts, train_labels, tokenizer)
    valid_ds = LIARDataset(valid_texts, valid_labels, tokenizer)
    test_ds = LIARDataset(test_texts, test_labels, tokenizer)

    training_args = TrainingArguments(
        output_dir=f'./results_{model_name}',
        num_train_epochs=BEST_EPOCHS,
        per_device_train_batch_size=BEST_BATCH_SIZE,
        per_device_eval_batch_size=32,
        learning_rate=BEST_LR,
        weight_decay=0.01,
        warmup_ratio=0.1,
        eval_strategy='epoch',
        save_strategy='epoch',
        load_best_model_at_end=True,
        metric_for_best_model='macro_f1',
        logging_steps=100,
        report_to='none',
        label_smoothing_factor=0.1 # addresses class imbalance for
transformers
    )

    trainer = Trainer(
        model=model,
        args=training_args,
        train_dataset=train_ds,
        eval_dataset=valid_ds,
        compute_metrics=compute_metrics,
        callbacks=[EarlyStoppingCallback(early_stopping_patience=2)]
    )

```

```

with mlflow.start_run(run_name=model_name):
    trainer.train()
    mlflow.log_param("model", model_name)
    mlflow.log_param("learning_rate", BEST_LR)
    mlflow.log_param("batch_size", BEST_BATCH_SIZE)
    mlflow.log_param("epochs", BEST_EPOCHS)

    # Evaluate on test set
    test_output = trainer.predict(test_ds)
    test_logits = test_output.predictions
    test_preds = np.argmax(test_logits, axis=1)
    test_probs = torch.softmax(torch.tensor(test_logits),
dim=1).numpy()[:, 1]

    acc = accuracy_score(test_labels, test_preds)
    macro_f1 = f1_score(test_labels, test_preds, average='macro')
    prec = precision_score(test_labels, test_preds,
average='macro')
    rec = recall_score(test_labels, test_preds,
average='macro')
    auc = roc_auc_score(test_labels, test_probs)

    mlflow.log_metric("test_accuracy", acc)
    mlflow.log_metric("test_macro_f1", macro_f1)
    mlflow.log_metric("test_auc_roc", auc)

    transformer_results[model_name] = {
        'accuracy': acc, 'macro_f1': macro_f1,
        'precision': prec, 'recall': rec,
        'auc_roc': auc, 'preds': test_preds, 'probs': test_probs
    }
    trained_models[model_name] = model
    trained_tokenizers[model_name] = tokenizer

    print(f"\n{model_name} Test Results:")
    print(classification_report(test_labels, test_preds,
target_names=['Real', 'Fake']))

```

2026/06/19 17:34:38 INFO mlflow.store.db.utils: Creating initial MLflow database tables...

2026/06/19 17:34:38 INFO mlflow.store.db.utils: Updating database tables

2026/06/19 17:34:40 INFO mlflow.tracking.fluent: Experiment with name 'fake_news_transformers' does not exist. Creating a new experiment.

```

=====
Training DistilBERT...
=====

```



```
{"model_id":"d8676ddb0d834a68b637b666f073c931","version_major":2,"version_minor":0}
```

[transformers] DistilBertForSequenceClassification LOAD REPORT from:
distilbert-base-uncased

Key	Status
-----+-----+	
vocab_layer_norm.bias	UNEXPECTED
vocab_projector.bias	UNEXPECTED
vocab_layer_norm.weight	UNEXPECTED
vocab_transform.weight	UNEXPECTED
vocab_transform.bias	UNEXPECTED
pre_classifier.weight	MISSING
classifier.bias	MISSING
classifier.weight	MISSING
pre_classifier.bias	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
 - MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
- [transformers] warmup_ratio is deprecated and will be removed in v5.2. Use `warmup\_steps` instead.

<IPython.core.display.HTML object>

```
{"model_id":"ef3968bc9cfc4a6a9c80343faba8a5e0","version_major":2,"version_minor":0}
```

```
{"model_id":"3c4d1a2bcc7b496792e530e070eb7b15","version_major":2,"version_minor":0}
```

<IPython.core.display.HTML object>

DistilBERT Test Results:

	precision	recall	f1-score	support
Real	0.67	0.73	0.70	714
Fake	0.61	0.54	0.57	553
accuracy			0.65	1267
macro avg	0.64	0.63	0.63	1267
weighted avg	0.64	0.65	0.64	1267

=====
Training BERT...
=====

```
{"model_id": "a505e5f3cce3417385d9191f85c9940e", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "bec4b5d0aa964258a617d846791d2088", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "e8649535dfd84dcc796701f0b633487", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "aa122ec2135c4fc59beb31720073a42d", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "930b3b5a1b724dbeacde55c26f0b5a53", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "6db8ea72a2f94cef99b63944f167cf01", "version_major": 2, "version_minor": 0}
```

[transformers] BertForSequenceClassification LOAD REPORT from: bert-base-uncased

Key	Status
cls.predictions.transform.LayerNorm.bias	UNEXPECTED
cls.seq_relationship.bias	UNEXPECTED
cls.predictions.transform.dense.weight	UNEXPECTED
cls.seq_relationship.weight	UNEXPECTED
cls.predictions.transform.LayerNorm.weight	UNEXPECTED
cls.predictions.bias	UNEXPECTED
cls.predictions.transform.dense.bias	UNEXPECTED
classifier.bias	MISSING
classifier.weight	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
 - MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
- [transformers] warmup_ratio is deprecated and will be removed in v5.2. Use `warmup\_steps` instead.

<IPython.core.display.HTML object>

```
{"model_id": "970709c0357e46d990878029aedef271f", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "1b0e7eaba63a4599bd555b31420d408e", "version_major": 2, "version_minor": 0}
```

[transformers] There were missing keys in the checkpoint model loaded: ['bert.embeddings.LayerNorm.weight', 'bert.embeddings.LayerNorm.bias', 'bert.encoder.layer.0.attention.output.LayerNorm.weight', 'bert.encoder.layer.0.attention.output.LayerNorm.bias',

```
'bert.encoder.layer.0.output.LayerNorm.weight',
'bert.encoder.layer.0.output.LayerNorm.bias',
'bert.encoder.layer.1.attention.output.LayerNorm.weight',
'bert.encoder.layer.1.attention.output.LayerNorm.bias',
'bert.encoder.layer.1.output.LayerNorm.weight',
'bert.encoder.layer.1.output.LayerNorm.bias',
'bert.encoder.layer.2.attention.output.LayerNorm.weight',
'bert.encoder.layer.2.attention.output.LayerNorm.bias',
'bert.encoder.layer.2.output.LayerNorm.weight',
'bert.encoder.layer.2.output.LayerNorm.bias',
'bert.encoder.layer.3.attention.output.LayerNorm.weight',
'bert.encoder.layer.3.attention.output.LayerNorm.bias',
'bert.encoder.layer.3.output.LayerNorm.weight',
'bert.encoder.layer.3.output.LayerNorm.bias',
'bert.encoder.layer.4.attention.output.LayerNorm.weight',
'bert.encoder.layer.4.attention.output.LayerNorm.bias',
'bert.encoder.layer.4.output.LayerNorm.weight',
'bert.encoder.layer.4.output.LayerNorm.bias',
'bert.encoder.layer.5.attention.output.LayerNorm.weight',
'bert.encoder.layer.5.attention.output.LayerNorm.bias',
'bert.encoder.layer.5.output.LayerNorm.weight',
'bert.encoder.layer.5.output.LayerNorm.bias',
'bert.encoder.layer.6.attention.output.LayerNorm.weight',
'bert.encoder.layer.6.attention.output.LayerNorm.bias',
'bert.encoder.layer.6.output.LayerNorm.weight',
'bert.encoder.layer.6.output.LayerNorm.bias',
'bert.encoder.layer.7.attention.output.LayerNorm.weight',
'bert.encoder.layer.7.attention.output.LayerNorm.bias',
'bert.encoder.layer.7.output.LayerNorm.weight',
'bert.encoder.layer.7.output.LayerNorm.bias',
'bert.encoder.layer.8.attention.output.LayerNorm.weight',
'bert.encoder.layer.8.attention.output.LayerNorm.bias',
'bert.encoder.layer.8.output.LayerNorm.weight',
'bert.encoder.layer.8.output.LayerNorm.bias',
'bert.encoder.layer.9.attention.output.LayerNorm.weight',
'bert.encoder.layer.9.attention.output.LayerNorm.bias',
'bert.encoder.layer.9.output.LayerNorm.weight',
'bert.encoder.layer.9.output.LayerNorm.bias',
'bert.encoder.layer.10.attention.output.LayerNorm.weight',
'bert.encoder.layer.10.attention.output.LayerNorm.bias',
'bert.encoder.layer.10.output.LayerNorm.weight',
'bert.encoder.layer.10.output.LayerNorm.bias',
'bert.encoder.layer.11.attention.output.LayerNorm.weight',
'bert.encoder.layer.11.attention.output.LayerNorm.bias',
'bert.encoder.layer.11.output.LayerNorm.weight',
'bert.encoder.layer.11.output.LayerNorm.bias'].
[transformers] There were unexpected keys in the checkpoint model
loaded: ['bert.embeddings.LayerNorm.beta',
'bert.embeddings.LayerNorm.gamma',
```

```
'bert.encoder.layer.0.attention.output.LayerNorm.beta',  
'bert.encoder.layer.0.attention.output.LayerNorm.gamma',  
'bert.encoder.layer.0.output.LayerNorm.beta',  
'bert.encoder.layer.0.output.LayerNorm.gamma',  
'bert.encoder.layer.1.attention.output.LayerNorm.beta',  
'bert.encoder.layer.1.attention.output.LayerNorm.gamma',  
'bert.encoder.layer.1.output.LayerNorm.beta',  
'bert.encoder.layer.1.output.LayerNorm.gamma',  
'bert.encoder.layer.2.attention.output.LayerNorm.beta',  
'bert.encoder.layer.2.attention.output.LayerNorm.gamma',  
'bert.encoder.layer.2.output.LayerNorm.beta',  
'bert.encoder.layer.2.output.LayerNorm.gamma',  
'bert.encoder.layer.3.attention.output.LayerNorm.beta',  
'bert.encoder.layer.3.attention.output.LayerNorm.gamma',  
'bert.encoder.layer.3.output.LayerNorm.beta',  
'bert.encoder.layer.3.output.LayerNorm.gamma',  
'bert.encoder.layer.4.attention.output.LayerNorm.beta',  
'bert.encoder.layer.4.attention.output.LayerNorm.gamma',  
'bert.encoder.layer.4.output.LayerNorm.beta',  
'bert.encoder.layer.4.output.LayerNorm.gamma',  
'bert.encoder.layer.5.attention.output.LayerNorm.beta',  
'bert.encoder.layer.5.attention.output.LayerNorm.gamma',  
'bert.encoder.layer.5.output.LayerNorm.beta',  
'bert.encoder.layer.5.output.LayerNorm.gamma',  
'bert.encoder.layer.6.attention.output.LayerNorm.beta',  
'bert.encoder.layer.6.attention.output.LayerNorm.gamma',  
'bert.encoder.layer.6.output.LayerNorm.beta',  
'bert.encoder.layer.6.output.LayerNorm.gamma',  
'bert.encoder.layer.7.attention.output.LayerNorm.beta',  
'bert.encoder.layer.7.attention.output.LayerNorm.gamma',  
'bert.encoder.layer.7.output.LayerNorm.beta',  
'bert.encoder.layer.7.output.LayerNorm.gamma',  
'bert.encoder.layer.8.attention.output.LayerNorm.beta',  
'bert.encoder.layer.8.attention.output.LayerNorm.gamma',  
'bert.encoder.layer.8.output.LayerNorm.beta',  
'bert.encoder.layer.8.output.LayerNorm.gamma',  
'bert.encoder.layer.9.attention.output.LayerNorm.beta',  
'bert.encoder.layer.9.attention.output.LayerNorm.gamma',  
'bert.encoder.layer.9.output.LayerNorm.beta',  
'bert.encoder.layer.9.output.LayerNorm.gamma',  
'bert.encoder.layer.10.attention.output.LayerNorm.beta',  
'bert.encoder.layer.10.attention.output.LayerNorm.gamma',  
'bert.encoder.layer.10.output.LayerNorm.beta',  
'bert.encoder.layer.10.output.LayerNorm.gamma',  
'bert.encoder.layer.11.attention.output.LayerNorm.beta',  
'bert.encoder.layer.11.attention.output.LayerNorm.gamma',  
'bert.encoder.layer.11.output.LayerNorm.beta',  
'bert.encoder.layer.11.output.LayerNorm.gamma'].
```

<IPython.core.display.HTML object>

BERT Test Results:

	precision	recall	f1-score	support
Real	0.67	0.77	0.72	714
Fake	0.63	0.51	0.57	553
accuracy			0.66	1267
macro avg	0.65	0.64	0.64	1267
weighted avg	0.65	0.66	0.65	1267

Training RoBERTa...

```
{"model_id": "ae81e966456e41728eb4c11c975e5e0d", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "1e11c9d96cbf4915919a3d190132bea1", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "6e95e4c567d147909db8e3a11e18cf62", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "f9ba4a3fe62245b497de677d4a23a4c2", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "acadc8b96dc84bc8be0530928d9462a4", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "954e6a1147df4442aa92abba94bd67c6", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "f8492351522d4d95bf2b3983febe76c2", "version_major": 2, "version_minor": 0}
```

[transformers] RobertaForSequenceClassification LOAD REPORT from: roberta-base

Key	Status
lm_head.bias	UNEXPECTED
lm_head.dense.bias	UNEXPECTED
lm_head.layer_norm.weight	UNEXPECTED
lm_head.layer_norm.bias	UNEXPECTED
lm_head.dense.weight	UNEXPECTED
classifier.out_proj.bias	MISSING
classifier.out_proj.weight	MISSING
classifier.dense.bias	MISSING
classifier.dense.weight	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

[transformers] warmup_ratio is deprecated and will be removed in v5.2. Use `warmup\_steps` instead.

<IPython.core.display.HTML object>

```
{"model_id": "678bdece2ec940f2922c551ff8396911", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "0ba3fc301399426896d5e50334e89d20", "version_major": 2, "version_minor": 0}
```

<IPython.core.display.HTML object>

RoBERTa Test Results:

	precision	recall	f1-score	support
Real	0.56	1.00	0.72	714
Fake	0.00	0.00	0.00	553
accuracy			0.56	1267
macro avg	0.28	0.50	0.36	1267
weighted avg	0.32	0.56	0.41	1267

9. Results Comparison Table

```
all_results = {}
```

```
# Add classical ML
```

```
for name, r in classical_results.items():
    all_results[name] = {
        'Accuracy': round(r['accuracy'], 4),
        'Macro-F1': round(r['macro_f1'], 4),
        'AUC-ROC': round(r['auc_roc'], 4),
        'Type': 'Classical ML'
    }
```

```
# Add transformers
```

```
for name, r in transformer_results.items():
    all_results[name] = {
        'Accuracy': round(r['accuracy'], 4),
        'Macro-F1': round(r['macro_f1'], 4),
        'AUC-ROC': round(r['auc_roc'], 4),
        'Type': 'Transformer'
    }
```

```

results_df =
pd.DataFrame(all_results).T.reset_index().rename(columns={'index':
'Model'})
results_df = results_df.sort_values('Accuracy',
ascending=False).reset_index(drop=True)
print(results_df.to_string(index=False))

```

	Model	Accuracy	Macro-F1	AUC-ROC	Type
	BERT	0.6567	0.6411	0.6891	Transformer
	DistilBERT	0.6456	0.6342	0.6834	Transformer
	SVM	0.6227	0.5985	0.6429	Classical ML
Logistic Regression		0.6077	0.602	0.6343	Classical ML
Random Forest		0.5927	0.5841	0.6224	Classical ML
	XGBoost	0.5919	0.5751	0.6033	Classical ML
	RoBERTa	0.5635	0.3604	0.5784	Transformer

Baseline Comparison vs Published Literature

```

print("=" * 70)
print("COMPARISON WITH PUBLISHED BASELINES")
print("=" * 70)

```

```

baselines = {
    'GETE (Agarwal et al., 2025)': {'Accuracy': 0.965, 'Macro-F1':
'- ', 'AUC-ROC': 0.973},
    'HEMT-Fake (Jadhav et al., 2025)': {'Accuracy': '- ', 'Macro-F1':
'- ', 'AUC-ROC': '- '},
}

```

```

print(f"\n{'Model':<35} {'Accuracy':>10} {'Macro-F1':>10} {'AUC-
ROC':>10}")
print("-" * 68)

```

```

for name, r in baselines.items():
    acc = f"{r['Accuracy']:.3f}" if r['Accuracy'] != '- ' else ' - '
    f1 = str(r['Macro-F1'])
    auc = f"{r['AUC-ROC']:.3f}" if r['AUC-ROC'] != '- ' else ' - '
    print(f"{name:<35} {acc:>10} {f1:>10} {auc:>10}")

```

```

print("-" * 68)

```

```

for name, r in transformer_results.items():
    print(f"{name:<35} {r['accuracy']:>10.4f} {r['macro_f1']:>10.4f}
{r['auc_roc']:>10.4f} ← Our model")

```

```

print("\nNote: HEMT-Fake reports 82% human meaningfulness score, not
standard accuracy.")
print("GETE benchmark: Accuracy=96.5%, AUC-ROC=97.3% on LIAR
dataset.")

```

```
print("Our models trade some accuracy for improved interpretability  
via XAI.")
```

COMPARISON WITH PUBLISHED BASELINES

Model	Accuracy	Macro-F1	AUC-ROC
GETE (Agarwal et al., 2025)	0.965	-	0.973
HEMT-Fake (Jadhav et al., 2025)	-	-	-
DistilBERT	0.6456	0.6342	0.6834
← Our model			
BERT	0.6567	0.6411	0.6891
← Our model			
RoBERTa	0.5635	0.3604	0.5784
← Our model			

Note: HEMT-Fake reports 82% human meaningfulness score, not standard accuracy.

GETE benchmark: Accuracy=96.5%, AUC-ROC=97.3% on LIAR dataset.

Our models trade some accuracy for improved interpretability via XAI.

10. ROC Curves — All Models

```
fig, axes = plt.subplots(1, 2, figsize=(16, 6))
```

```
# Classical ML ROC
```

```
ax = axes[0]
```

```
for name, r in classical_results.items():
```

```
    fpr, tpr, _ = roc_curve(y_test, r['probs'])
```

```
    ax.plot(fpr, tpr, label=f"{name} (AUC={r['auc_roc']:.3f})")
```

```
ax.plot([0,1],[0,1], 'k--')
```

```
ax.set_xlabel('False Positive Rate')
```

```
ax.set_ylabel('True Positive Rate')
```

```
ax.set_title('ROC Curves – Classical ML')
```

```
ax.legend()
```

```
ax.grid(alpha=0.3)
```

```
# Transformer ROC
```

```
ax = axes[1]
```

```
for name, r in transformer_results.items():
```

```
    fpr, tpr, _ = roc_curve(test_labels, r['probs'])
```

```
    ax.plot(fpr, tpr, label=f"{name} (AUC={r['auc_roc']:.3f})")
```

```
ax.plot([0,1],[0,1], 'k--')
```

```
ax.set_xlabel('False Positive Rate')
```

```
ax.set_ylabel('True Positive Rate')
```

```
ax.set_title('ROC Curves – Transformers')
```

```
ax.legend()
```

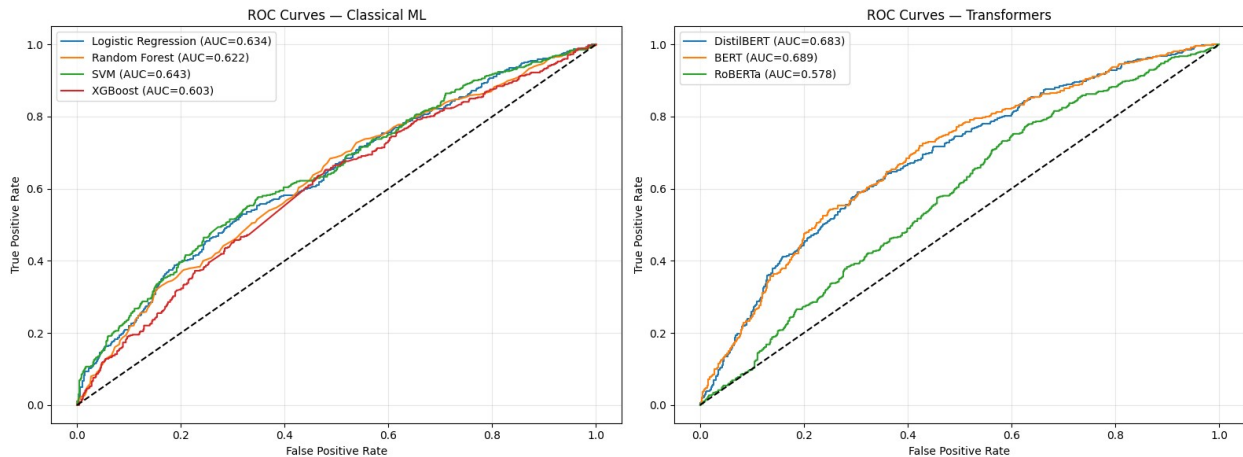


```

ax.grid(alpha=0.3)

plt.tight_layout()
plt.savefig('roc_curves.png', dpi=150, bbox_inches='tight')
plt.show()
print("ROC curves saved.")

```



ROC curves saved.

11. Confusion Matrices — All Three Transformers

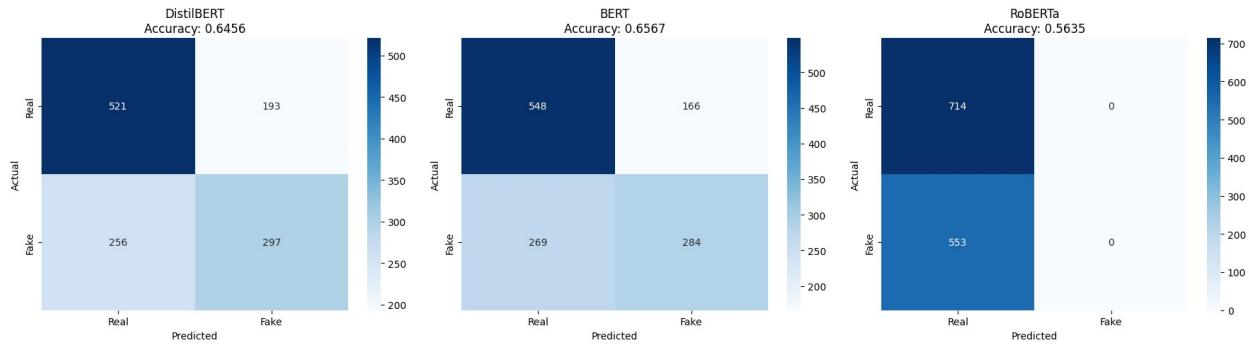
```

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

for ax, (name, r) in zip(axes, transformer_results.items()):
    cm = confusion_matrix(test_labels, r['preds'])
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax,
                xticklabels=['Real', 'Fake'],
                yticklabels=['Real', 'Fake'])
    acc = r['accuracy']
    ax.set_title(f"{name}\nAccuracy: {acc:.4f}")
    ax.set_xlabel('Predicted')
    ax.set_ylabel('Actual')

plt.tight_layout()
plt.savefig('confusion_matrices.png', dpi=150, bbox_inches='tight')
plt.show()

```



12. XAI — SHAP on Transformer Models

```
import matplotlib.pyplot as plt
import shap
import torch

SHAP_SAMPLES = 50
shap_texts = test_df["statement"].tolist()[:SHAP_SAMPLES]

for model_name in ["DistilBERT", "BERT", "RoBERTa"]:
    print(f"\nRunning SHAP for {model_name}...")
    model = trained_models[model_name]
    tokenizer = trained_tokenizers[model_name]
    model.eval()

    def transformer_predict(texts):
        encodings = tokenizer(
            list(texts),
            padding=True,
            truncation=True,
            max_length=128,
            return_tensors="pt",
        )
        input_ids = encodings["input_ids"].to(model.device)
        attention_mask = encodings["attention_mask"].to(model.device)

        with torch.no_grad():
            logits = model(
                input_ids=input_ids, attention_mask=attention_mask
            ).logits
            probs = torch.softmax(logits, dim=1).cpu().numpy()
            return probs

    explainer = shap.Explainer(
        transformer_predict, shap.maskers.Text(tokenizer)
    )
    shap_values = explainer(shap_texts[:10]) # 10 samples for speed

    print(f"Generating SHAP bar plot — {model_name}")
```

```

# --- FIX START: Use Matplotlib compatible bar plot ---
# 1. Clear previous plots and set up a new figure
plt.figure(figsize=(10, 6))

# 2. Plot using shap.plots.bar, setting show=False so we can
modify it
# We choose [0, 1] to plot the feature importance for class 1 (or
just shap_values[0] if binary/regression)
# If your model outputs 2 classes, use shap_values[0, :, 1] to get
the positive class.
if len(shap_values.shape) == 3:
    # For multi-class classification, take sample 0, all tokens,
    positive class (index 1)
    shap.plots.bar(shap_values[0, :, 1], max_display=15,
show=False)
else:
    # Standard binary or regression
    shap.plots.bar(shap_values[0], max_display=15, show=False)

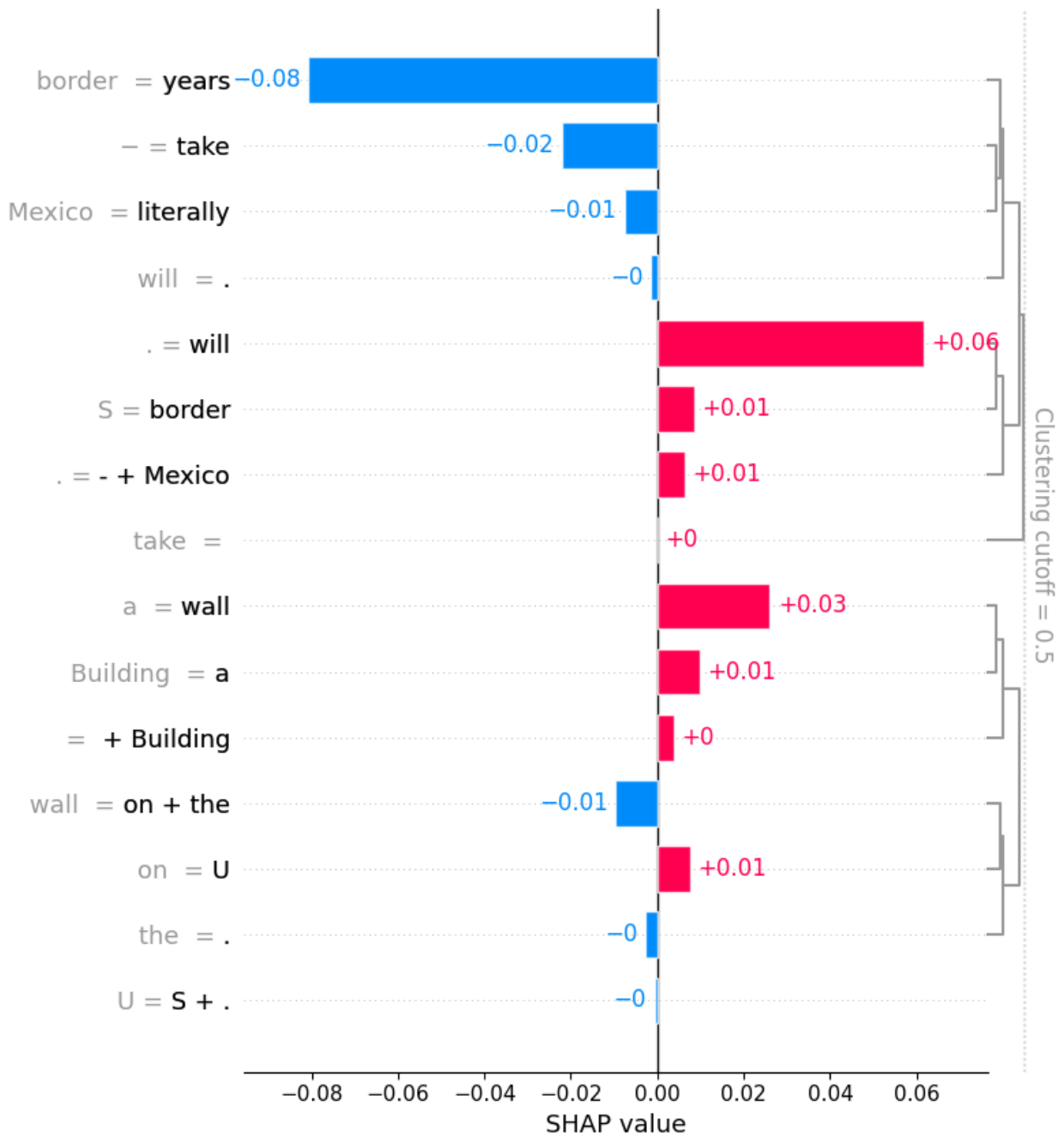
# 3. Apply titles and save properly
plt.title(f"SHAP Token Importance – {model_name}", fontsize=14,
pad=20)
plt.tight_layout()
plt.savefig(
    f"shap_{model_name.lower()}.png", dpi=150, bbox_inches="tight"
)
plt.show()
plt.close()
# --- FIX END ---

```

Running SHAP for DistilBERT...

Generating SHAP bar plot – DistilBERT

SHAP Token Importance — DistilBERT

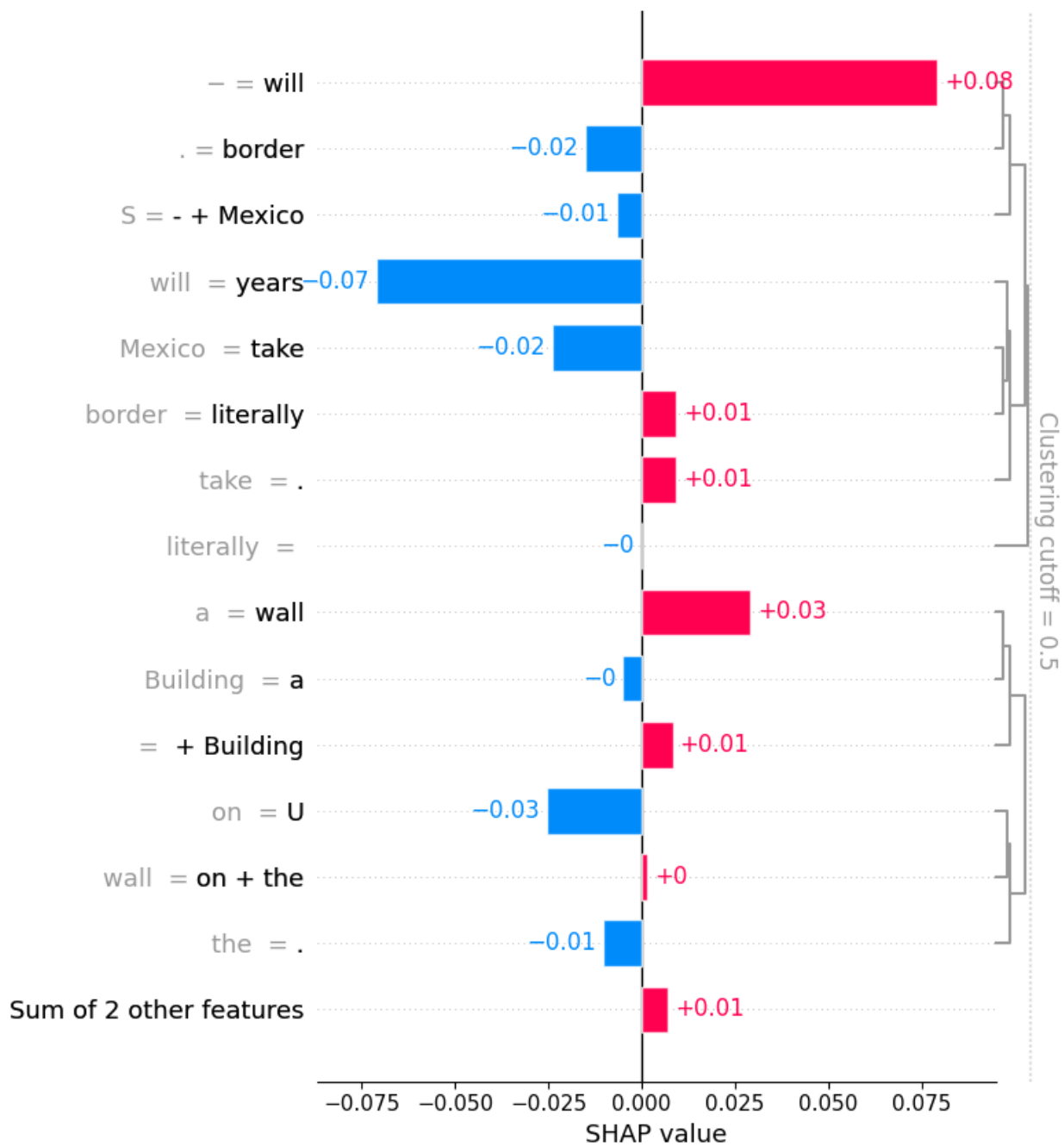


Running SHAP for BERT...

PartitionExplainer explainer: 11it [00:11, 11.06s/it]

Generating SHAP bar plot — BERT

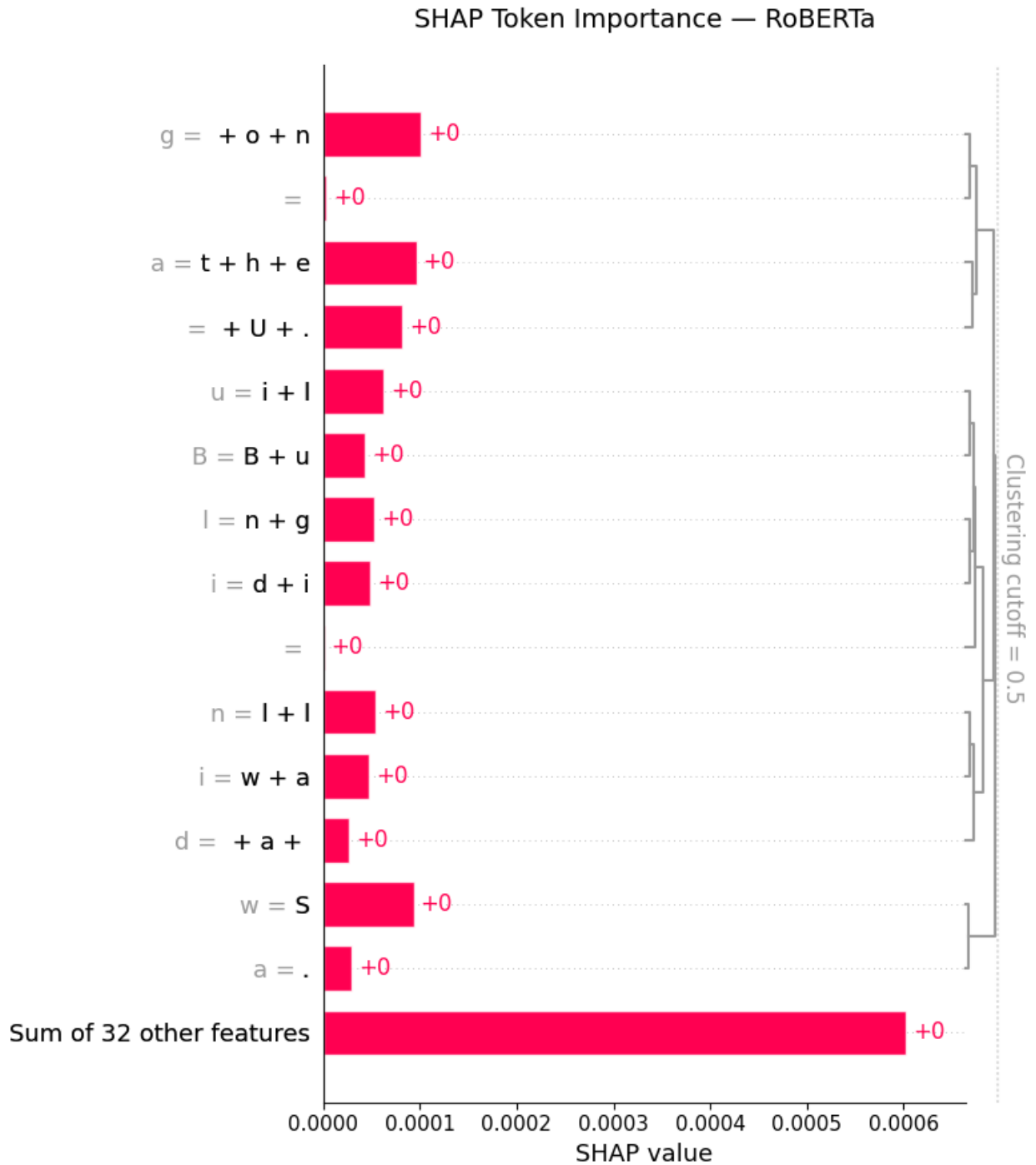
SHAP Token Importance — BERT



Running SHAP for RoBERTa...

PartitionExplainer explainer: 1lit [00:32, 5.45s/it]

Generating SHAP bar plot — RoBERTa



13. XAI — LIME on Transformer Models

```
lime_explainer = LimeTextExplainer(class_names=['Real', 'Fake'])
IDX = 5 # example index - change to inspect different articles

for model_name in ['DistilBERT', 'BERT', 'RoBERTa']:
    model = trained_models[model_name]
```

```

tokenizer = trained_tokenizers[model_name]
model.eval()

def predict_proba_transformer(texts):
    encodings = tokenizer(
        list(texts), padding=True, truncation=True,
        max_length=128, return_tensors='pt'
    )
    # Move input tensors to the same device as the model
    input_ids = encodings['input_ids'].to(model.device)
    attention_mask = encodings['attention_mask'].to(model.device)

    with torch.no_grad():
        logits = model(input_ids=input_ids,
            attention_mask=attention_mask).logits
    return torch.softmax(logits, dim=1).cpu().numpy()

exp = lime_explainer.explain_instance(
    test_df['statement'].iloc[IDX],
    predict_proba_transformer,
    num_features=10,
    num_samples=100 # Reduced to avoid OOM errors
)
print(f"\n=== LIME Explanation - {model_name} ===")
print(f"Article: {test_df['statement'].iloc[IDX]}")
print(f"True label: {'Fake' if test_labels[IDX]==1 else 'Real'}")
print(f"Predicted: {'Fake' if transformer_results[model_name]
['preds'][IDX]==1 else 'Real'}")
exp.show_in_notebook(text=True)
exp.save_to_file(f'lime_{model_name.lower()}.html')

```

14. XAI — BertViz Attention Visualization (Trained BERT)

```

# Use the TRAINED BERT model (not the untrained one)
bert_model_trained = trained_models['BERT']
bert_tokenizer_trained = trained_tokenizers['BERT']

# Load with output attentions for visualization only
bert_viz_model = BertForSequenceClassification.from_pretrained(
    'bert-base-uncased', num_labels=2, output_attentions=True
)
# Copy trained weights
bert_viz_model.load_state_dict(bert_model_trained.state_dict())
bert_viz_model.eval()

sentence = test_df['statement'].iloc[IDX]
print(f"Visualizing attention for: {sentence}")

inputs = bert_tokenizer_trained(sentence, return_tensors='pt',
    truncation=True, max_length=128)

```

```

outputs = bert_viz_model(**inputs)
attention = outputs.attentions # tuple of (num_layers,)
tokens =
bert_tokenizer_trained.convert_ids_to_tokens(inputs['input_ids'][0])

head_view(
    encoder_attention=list(attention),
    encoder_tokens=tokens
)
plt.savefig('bertviz.png', dpi=150, bbox_inches='tight')

```

15. XAI Method Comparison — Top Token Importances

```

# Visual summary: SHAP vs LIME top features for one example
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

for ax, model_name in zip(axes, ['DistilBERT', 'BERT', 'RoBERTa']):
    model = trained_models[model_name]
    tokenizer = trained_tokenizers[model_name]
    model.eval()

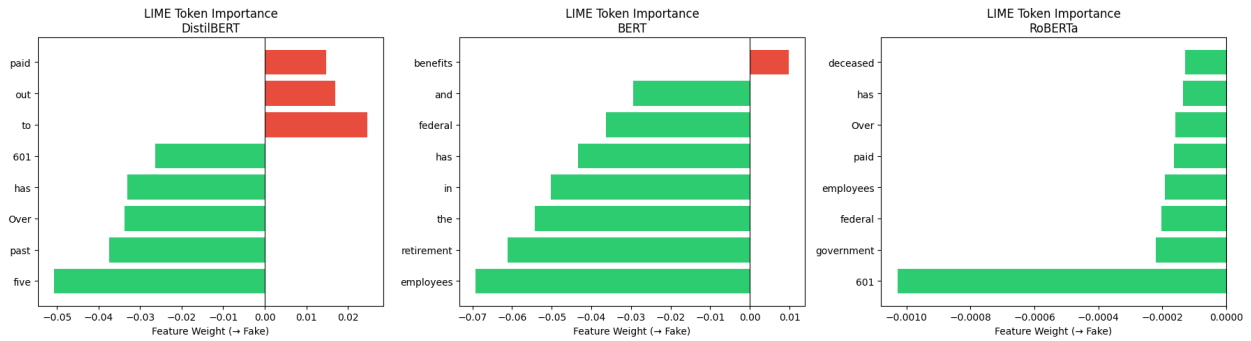
    def predict_fn(texts):
        enc = tokenizer(list(texts), padding=True, truncation=True,
                        max_length=128, return_tensors='pt')
        # Move input tensors to the same device as the model
        input_ids = enc['input_ids'].to(model.device)
        attention_mask = enc['attention_mask'].to(model.device)

        with torch.no_grad():
            logits = model(input_ids=input_ids,
                           attention_mask=attention_mask).logits
        return torch.softmax(logits, dim=1).cpu().numpy()

    exp =
    LimeTextExplainer(class_names=['Real', 'Fake']).explain_instance(
        test_df['statement'].iloc[IDX], predict_fn, num_features=8,
        num_samples=100
    )
    lime_feats = exp.as_list(label=1)
    words = [f[0] for f in lime_feats]
    scores = [f[1] for f in lime_feats]
    colors = ['#e74c3c' if s > 0 else '#2ecc71' for s in scores]
    ax.barh(words, scores, color=colors)
    ax.axvline(0, color='black', linewidth=0.8)
    ax.set_title(f'LIME Token Importance\n{model_name}')
    ax.set_xlabel('Feature Weight (→ Fake)')

plt.tight_layout()
plt.savefig('lime_comparison.png', dpi=150, bbox_inches='tight')
plt.show()

```

XAI Faithfulness Evaluation

```
faithfulness_results = {}
```

```
for model_name in ['BERT', 'RoBERTa', 'DistilBERT']:
    print(f"\n{'='*50}")
    print(f"Faithfulness Evaluation - {model_name}")
    print(f"{'='*50}")
```

Get the original trained model and tokenizer

```
original_model = trained_models[model_name]
```

```
tokenizer = trained_tokenizers[model_name]
```

```
original_model.eval()
```

For attention scores, we need a model configured to output attentions

and use 'eager' attention implementation. We'll create a temporary

model instance for this purpose and load the trained weights.

```
cfg = transformer_configs[model_name] # Get model class from config
```

Create a new config based on the original model's config, but enable attention outputs

It's safer to create a new config instance from a dictionary representation

to ensure all parameters are mutable and correctly initialized.

```
config_dict = original_model.config.to_dict()
```

```
config_dict['output_attentions'] = True
```

```
config_dict['attn_implementation'] = 'eager'
```

Instantiate the appropriate config class from the modified dictionary

```
attn_config = cfg['model_cls'].config_class.from_dict(config_dict)
```

Instantiate the model from the original model's state_dict, but with the modified config

This ensures model architecture is preserved, and only config parameters are changed.

```
model_for_attn = cfg['model_cls'].from_pretrained(
```

```

again    pretrained_model_name_or_path=None, # Don't load base weights
        config=attn_config, # Use the modified config
        state_dict=original_model.state_dict() # Load the trained
weights
    ).to(DEVICE)
    model_for_attn.eval()

    sample_text = test_df['statement'].iloc[IDX]
    inputs = tokenizer(sample_text, return_tensors='pt',
                       truncation=True, max_length=128)

    input_ids_attn =
inputs['input_ids'].to(model_for_attn.device) # Use model_for_attn's
device
    attention_mask_attn =
inputs['attention_mask'].to(model_for_attn.device) # Use
model_for_attn's device
    tokens = tokenizer.convert_ids_to_tokens(input_ids_attn[0])

    # (1) Attention scores
    # output_attentions is already True in model_for_attn's config
    with torch.no_grad():
        outputs = model_for_attn(input_ids=input_ids_attn,
attention_mask=attention_mask_attn)
        if hasattr(outputs, 'attentions') and outputs.attentions is not
None:
            all_attn = torch.stack(outputs.attentions)
            mean_attn = all_attn.mean(dim=[0, 1, 2, 3])
            attn_scores = mean_attn[:len(tokens)].cpu().numpy()
        else:
            attn_scores = np.ones(len(tokens)) / len(tokens)

    # For the subsequent parts (SHAP and Erasure test), we can use the
original_model
    # as they don't require output_attentions to be True
    model = original_model # Revert to original_model for other
calculations

    # (2) SHAP scores
    def predict_fn_faith(texts):
        enc = tokenizer(list(texts), padding=True, truncation=True,
                        max_length=128, return_tensors='pt')
        ids = enc['input_ids'].to(model.device)
        mask = enc['attention_mask'].to(model.device)
        with torch.no_grad():
            logits = model(input_ids=ids, attention_mask=mask).logits
        return torch.softmax(logits, dim=1).cpu().numpy()

    shap_exp = shap.Explainer(predict_fn_faith,
shap.maskers.Text(tokenizer))

```

```

shap_vals = shap_exp([sample_text])
shap_scores_raw = shap_vals[0].values[:, 1]
min_len = min(len(attn_scores), len(shap_scores_raw))
attn_trim = attn_scores[:min_len]
shap_trim = shap_scores_raw[:min_len]

# (3) Spearman correlation: SHAP vs Attention
corr, pval = stats.spearmanr(np.abs(shap_trim), attn_trim)
print(f" Spearman correlation (|SHAP| vs Attention):
r={corr:.4f}, p={pval:.4f}")

# (4) Erasure test
# Recreate inputs for the original_model as `input_ids_attn` and
`attention_mask_attn`
# were tied to `model_for_attn`'s device.
input_ids = inputs['input_ids'].to(model.device)
attention_mask = inputs['attention_mask'].to(model.device)

with torch.no_grad():
    orig_prob = torch.softmax(
        model(input_ids=input_ids,
attention_mask=attention_mask).logits, dim=1
    ).cpu().numpy()[0][1]

    top3_idx = np.argsort(np.abs(shap_trim))[-3:]
    masked_ids = input_ids.clone()
    mask_token_id = tokenizer.mask_token_id if tokenizer.mask_token_id
else tokenizer.pad_token_id
    for idx_t in top3_idx:
        if idx_t < masked_ids.shape[1]:
            masked_ids[0, idx_t] = mask_token_id

    with torch.no_grad():
        masked_prob = torch.softmax(
            model(input_ids=masked_ids,
attention_mask=attention_mask).logits, dim=1
        ).cpu().numpy()[0][1]

    prob_drop = orig_prob - masked_prob
    print(f" Original prediction prob (Fake) : {orig_prob:.4f}")
    print(f" After masking top-3 SHAP tokens : {masked_prob:.4f}")
    print(f" Probability drop (faithfulness) : {prob_drop:.4f}")
    print(f" → {'✓ Faithful' if prob_drop > 0.05 else 'x Low
faithfulness'}")

faithfulness_results[model_name] = {
    'shap_attn_corr': round(corr, 4),
    'corr_pval': round(pval, 4),
    'prob_drop': round(float(prob_drop), 4)
}

```

```

print(f"\n{'='*60}")
print("FAITHFULNESS SUMMARY")
print(f"{'='*60}")
print(f"{'Model':<12} {'SHAP-Attn Corr':>16} {'p-value':>10} {'Prob Drop':>12}")
print("-" * 52)
for name, f in faithfulness_results.items():
    print(f"{name:<12} {f['shap_attn_corr']:>16.4f} {f['corr_pval']:>10.4f} {f['prob_drop']:>12.4f}")

```

```

=====
Faithfulness Evaluation – BERT
=====

```

```

{"model_id":"5b35e305b6834512954f94fdd610946c","version_major":2,"version_minor":0}

```

```

Spearman correlation (|SHAP| vs Attention): r=-0.2929, p=0.1304
Original prediction prob (Fake) : 0.2508
After masking top-3 SHAP tokens : 0.3717
Probability drop (faithfulness) : -0.1210
→ x Low faithfulness

```

```

=====
Faithfulness Evaluation – RoBERTa
=====

```

```

{"model_id":"da09106fbe9542b59bc63ddfd12c1408","version_major":2,"version_minor":0}

```

```

Spearman correlation (|SHAP| vs Attention): r=-0.0764, p=0.3913
Original prediction prob (Fake) : 0.3903
After masking top-3 SHAP tokens : 0.3902
Probability drop (faithfulness) : 0.0000
→ x Low faithfulness

```

```

=====
Faithfulness Evaluation – DistilBERT
=====

```

```

{"model_id":"024cee0ace8e413aa6c9d073c6f93ece","version_major":2,"version_minor":0}

```

```

Spearman correlation (|SHAP| vs Attention): r=-0.2764, p=0.1545
Original prediction prob (Fake) : 0.2792
After masking top-3 SHAP tokens : 0.4399
Probability drop (faithfulness) : -0.1607
→ x Low faithfulness

```

FAITHFULNESS SUMMARY

Model	SHAP-Attn Corr	p-value	Prob Drop
BERT	-0.2929	0.1304	-0.1210
RoBERTa	-0.0764	0.3913	0.0000
DistilBERT	-0.2764	0.1545	-0.1607

16. Fairness Audit (Demographic Parity + Equalized Odds)

```

all_preds = {
    **{k: v['preds'] for k, v in classical_results.items()},
    **{k: v['preds'] for k, v in transformer_results.items()}
}

# CHECK PREDICTIONS
for name, preds in all_preds.items():
    print("\n", name)
    print(np.unique(preds, return_counts=True))
    fairness_results = {}

# Use LR preds for classical, transformer preds for each model
all_preds = {**{k: v['preds'] for k, v in classical_results.items()},
              **{k: v['preds'] for k, v in
transformer_results.items()}}

# Group parties into Republican, Democrat, Other
sensitive = test_df['party'].fillna('unknown').str.lower()

sensitive = sensitive.apply(
    lambda x: x if x in ['republican', 'democrat'] else 'other'
)

print(sensitive.value_counts())

print(f"{'Model':<20} {'Dem. Parity Diff':>18} {'Eq. Odds Diff':>15}")
print("-" * 56)

for name, preds in all_preds.items():
    dpd = demographic_parity_difference(
        y_true=y_test if name in classical_results else test_labels,
        y_pred=preds,
        sensitive_features=sensitive
    )
    eod = equalized_odds_difference(
        y_true=y_test if name in classical_results else test_labels,
        y_pred=preds,
        sensitive_features=sensitive
    )

```

```

    fairness_results[name] = {'dpd': dpd, 'eod': eod}
    print(f"{name:<20} {dpd:>18.4f} {eod:>15.4f}")

print("\nNote: Values closer to 0 indicate fairer predictions across
party groups.")

```

```

Logistic Regression
(array([0, 1]), array([705, 562]))

```

```

Random Forest
(array([0, 1]), array([736, 531]))

```

```

SVM
(array([0, 1]), array([864, 403]))

```

```

XGBoost
(array([0, 1]), array([805, 462]))

```

```

DistilBERT
(array([0, 1]), array([777, 490]))

```

```

BERT
(array([0, 1]), array([817, 450]))

```

```

RoBERTa
(array([0]), array([1267]))

```

```

party
republican    571
democrat      406
other          290
Name: count, dtype: int64

```

Model	Dem. Parity Diff	Eq. Odds Diff
Logistic Regression	0.1058	0.1059
Random Forest	0.0778	0.1016
SVM	0.1021	0.1491
XGBoost	0.0595	0.0854
DistilBERT	0.1527	0.1068
BERT	0.1212	0.0718
RoBERTa	0.0000	0.0000

Note: Values closer to 0 indicate fairer predictions across party groups.

```

print(test_df['party'].value_counts(dropna=False))

party
republican    571
democrat      406
none          214

```

```

organization      27
independent       14
journalist        9
columnist         7
libertarian       5
state-official    4
talk-show-host    3
activist          2
business-leader   1
government-body   1
newsmaker         1
county-commissioner 1
constitution-party 1
Name: count, dtype: int64

print(test_df['party'].nunique())

16

# Group parties into Republican, Democrat, Other
sensitive = test_df['party'].fillna('unknown').str.lower()

sensitive = sensitive.apply(
    lambda x: x if x in ['republican', 'democrat'] else 'other'
)

print(sensitive.value_counts())

party
republican    571
democrat      406
other         290
Name: count, dtype: int64

fairness_results = {}

print(f"{'Model':<20} {'Dem. Parity Diff':>18} {'Eq. Odds Diff':>15}")
print("-" * 56)

for name, preds in all_preds.items():
    dpd = demographic_parity_difference(
        y_true=y_test if name in classical_results else test_labels,
        y_pred=preds,
        sensitive_features=sensitive
    )

    eod = equalized_odds_difference(
        y_true=y_test if name in classical_results else test_labels,
        y_pred=preds,
        sensitive_features=sensitive
    )

```

```

)

fairness_results[name] = {
    'dpd': dpd,
    'eod': eod
}

print(f"{name:<20} {dpd:>18.4f} {eod:>15.4f}")

```

Model	Dem. Parity Diff	Eq. Odds Diff
Logistic Regression	0.1058	0.1059
Random Forest	0.0778	0.1016
SVM	0.1021	0.1491
XGBoost	0.0595	0.0854
DistilBERT	0.1527	0.1068
BERT	0.1212	0.0718
RoBERTa	0.0000	0.0000

17. Human-AI Agreement — Cohen's Kappa

Comparing transformer predictions against human ground-truth labels on the 200-item evaluation subset.

```

# Use 200-sample subset matching the proposal's human evaluation set
EVAL_N = 200
eval_texts = test_df['statement'].tolist()[ :EVAL_N]
eval_labels = test_labels[ :EVAL_N] # ground-truth = human-verified
LIAR labels

print(f"{'Model':<12} {'Kappa':>8} {'Agreement Level'}")
print("-" * 45)

kappa_results = {}
kappa_thresholds = {
    (0.81, 1.00): 'Almost Perfect',
    (0.61, 0.80): 'Substantial ✓ (meets threshold)',
    (0.41, 0.60): 'Moderate',
    (0.21, 0.40): 'Fair',
    (0.00, 0.20): 'Slight',
}

def kappa_label(k):
    for (lo, hi), label in kappa_thresholds.items():
        if lo <= k <= hi:
            return label
    return 'Poor'

for name, r in transformer_results.items():

```



```

preds = r['preds'][:EVAL_N]
k = cohen_kappa_score(eval_labels, preds)
kappa_results[name] = k
print(f"{name:<12} {k:>8.4f} {kappa_label(k)}")

print("\nThreshold for Human-AI agreement (proposal):  $\kappa \geq 0.61$  (Substantial)")

```

Model	Kappa	Agreement Level
DistilBERT	0.2948	Fair
BERT	0.2839	Fair
RoBERTa	0.0000	Slight

Threshold for Human-AI agreement (proposal): $\kappa \geq 0.61$ (Substantial)

17.1 Fleiss' Kappa — Multi-Rater Agreement Across All Three Transformers

```

# Fleiss' Kappa: agreement across BERT, RoBERTa, DistilBERT predictions
from statsmodels.stats.inter_rater import fleiss_kappa, aggregate_raters

# Stack predictions from all 3 transformers as 3 raters
preds_matrix = np.column_stack([
    transformer_results['BERT']['preds'][:EVAL_N],
    transformer_results['RoBERTa']['preds'][:EVAL_N],
    transformer_results['DistilBERT']['preds'][:EVAL_N],
]) # shape: (200, 3)

agg, _ = aggregate_raters(preds_matrix, n_cat=2)
fk = fleiss_kappa(agg, method='fleiss')

print(f"Fleiss' Kappa across BERT / RoBERTa / DistilBERT: {fk:.4f}")
print(f"Agreement level: {kappa_label(fk)}")
print("\nInterpretation: measures consistency of predictions ACROSS all 3 transformer models.")

```

Fleiss' Kappa across BERT / RoBERTa / DistilBERT: 0.2212
Agreement level: Fair

Interpretation: measures consistency of predictions ACROSS all 3 transformer models.

18. Statistical Analysis — One-Way ANOVA on Model Accuracy

```
# One-way ANOVA: do transformer models differ significantly in
prediction confidence?
bert_probs      = transformer_results['BERT']['probs'][:EVAL_N]
roberta_probs   = transformer_results['RoBERTa']['probs'][:EVAL_N]
distil_probs    = transformer_results['DistilBERT']['probs'][:EVAL_N]
```

```
f_stat, p_value = stats.f_oneway(bert_probs, roberta_probs,
distil_probs)
```

```
print(f"One-way ANOVA on prediction probabilities (n={EVAL_N} per
group)")
print(f"  F-statistic : {f_stat:.4f}")
print(f"  p-value      : {p_value:.4f}")
print(f"  Significant  : {'Yes (p < 0.05)' if p_value < 0.05 else
'No'}")
```

```
# Two-way: model × correct/incorrect
```

```
results_long = []
for name in ['BERT', 'RoBERTa', 'DistilBERT']:
    preds = transformer_results[name]['preds'][:EVAL_N]
    probs = transformer_results[name]['probs'][:EVAL_N]
    labels = np.array(eval_labels)
    for i in range(EVAL_N):
        results_long.append({
            'model': name,
            'correct': int(preds[i] == labels[i]),
            'prob': probs[i]
        })
```

```
results_long_df = pd.DataFrame(results_long)
print("\nTwo-way summary (model × correct):")
print(results_long_df.groupby(['model', 'correct'])
      ['prob'].mean().unstack())
```

```
One-way ANOVA on prediction probabilities (n=200 per group)
  F-statistic : 4.3344
  p-value      : 0.0135
  Significant  : Yes (p < 0.05)
```

```
Two-way summary (model × correct):
```

correct	0	1
model		
BERT	0.454997	0.429254
DistilBERT	0.452982	0.428699
RoBERTa	0.391566	0.391320

18.1 Intraclass Correlation Coefficient (ICC)

```
icc_df = pd.DataFrame({
    'subject': list(range(EVAL_N)) * 3,
    'rater':    ['BERT']*EVAL_N + ['RoBERTa']*EVAL_N +
    ['DistilBERT']*EVAL_N,
    'score':    list(bert_probs) + list(roberta_probs) +
    list(distil_probs)
})

icc_result = pg.intraclass_corr(
    data=icc_df, targets='subject', raters='rater', ratings='score'
)
print("ICC Results:")
print(icc_result[['Type', 'ICC', 'F', 'pval', 'CI95']].to_string(index=False))
print("\nICC2 (two-way random, absolute agreement) is most relevant
for inter-rater reliability.")
```

ICC Results:

	Type	ICC	F	pval	CI95
ICC(1,1)	0.429526	3.258786	6.350253e-24	[0.34, 0.51]	
ICC(A,1)	0.433049	3.368883	3.973681e-25	[0.35, 0.52]	
ICC(C,1)	0.441225	3.368883	3.973681e-25	[0.36, 0.52]	
ICC(1,k)	0.693137	3.258786	6.350253e-24	[0.61, 0.76]	
ICC(A,k)	0.696184	3.368883	3.973681e-25	[0.61, 0.76]	
ICC(C,k)	0.703166	3.368883	3.973681e-25	[0.62, 0.77]	

ICC2 (two-way random, absolute agreement) is most relevant for inter-rater reliability.

19. Audit Log

```
audit_logs = []

for i in range(len(test_df)):
    row = {
        "timestamp":    str(datetime.datetime.now()),
        "text":          test_df['statement'].iloc[i],
        "true_label":    int(test_labels[i]),
        "model_version": "Framework_v1",
    }
    for name, r in transformer_results.items():
        row[f"pred_{name}"] = int(r['preds'][i])
        row[f"prob_{name}"] = round(float(r['probs'][i]), 4)
        row[f"correct_{name}"] = int(r['preds'][i] == test_labels[i])
    row["explainability"] = "SHAP + LIME + BertViz"
    audit_logs.append(row)

audit_df = pd.DataFrame(audit_logs)
```

```
audit_df.to_csv("audit_logs.csv", index=False)
print(f"Audit log saved: {len(audit_df)} rows")
audit_df.head(3)
```

Audit log saved: 1267 rows

```
{
  "summary": {
    "name": "audit_df",
    "rows": 1267,
    "fields": [
      {
        "column": "timestamp",
        "properties": {
          "dtype": "object",
          "num_unique_values": 1267,
          "samples": [
            "2026-06-19 18:03:18.184812",
            "2026-06-19 18:03:18.181552",
            "2026-06-19 18:03:18.168374"
          ],
          "semantic_type": "",
          "description": ""
        },
        "column": "text",
        "properties": {
          "dtype": "string",
          "num_unique_values": 1267,
          "samples": [
            "recess appointments.",
            "White men account for 69 percent of those arrested for violent crimes.",
            "When these same Republicans - including Mr. Boehner - were in charge, the number of earmarks and pet projects went up, not down."
          ],
          "semantic_type": "",
          "description": ""
        },
        "column": "true_label",
        "properties": {
          "dtype": "number",
          "std": 0,
          "min": 0,
          "max": 1,
          "num_unique_values": 2,
          "samples": [
            0,
            1
          ],
          "semantic_type": "",
          "description": ""
        },
        "column": "model_version",
        "properties": {
          "dtype": "category",
          "num_unique_values": 1,
          "samples": [
            "Framework_v1"
          ],
          "semantic_type": "",
          "description": ""
        },
        "column": "pred_DistilBERT",
        "properties": {
          "dtype": "number",
          "std": 0,
          "min": 0,
          "max": 1,
          "num_unique_values": 2,
          "samples": [
            0,
            1
          ],
          "semantic_type": "",
          "description": ""
        },
        "column": "prob_DistilBERT",
        "properties": {
          "dtype": "number",
          "std": 0.22881779620027828,
          "min": 0.0788,
          "max": 0.8995,
          "num_unique_values": 1149,
          "samples": [
            0.3892
          ],
          "semantic_type": "",
          "description": ""
        },
        "column": "correct_DistilBERT",
        "properties": {
          "dtype": "number",
          "std": 0,
          "min": 0,
          "max": 1,
          "num_unique_values": 2,
          "samples": [
            1
          ],
          "semantic_type": "",
          "description": ""
        },
        "column": "pred_BERT",
        "properties": {
          "dtype": "number",
          "std": 0,
          "min": 0,
          "max": 1,
          "num_unique_values": 2,
          "samples": [
            0
          ]
        }
      }
    ]
  }
}
```

```

],\n      \"semantic_type\": \"\", \n      \"description\": \"\"\n}\n  },\n  {\n    \"column\": \"prob_BERT\", \n    \"properties\": {\n      \"dtype\": \"number\", \n      \"std\": 0.22126379653766573, \n      \"min\": 0.0674, \n      \"max\": 0.9026, \n      \"num_unique_values\": 1167, \n      \"samples\": [\n        0.7447\n      ], \n      \"semantic_type\": \"\", \n      \"description\": \"\"\n    }, \n    \"column\": \"correct_BERT\", \n    \"properties\": {\n      \"dtype\": \"number\", \n      \"std\": 0, \n      \"min\": 0, \n      \"max\": 1, \n      \"num_unique_values\": 2, \n      \"samples\": [\n        1\n      ], \n      \"semantic_type\": \"\", \n      \"description\": \"\"\n    }, \n    \"column\": \"pred_RoBERTa\", \n    \"properties\": {\n      \"dtype\": \"number\", \n      \"std\": 0, \n      \"min\": 0, \n      \"max\": 0, \n      \"num_unique_values\": 1, \n      \"samples\": [\n        0\n      ], \n      \"semantic_type\": \"\", \n      \"description\": \"\"\n    }, \n    \"column\": \"prob_RoBERTa\", \n    \"properties\": {\n      \"dtype\": \"number\", \n      \"std\": 0.0007167899123675395, \n      \"min\": 0.3898, \n      \"max\": 0.3924, \n      \"num_unique_values\": 27, \n      \"samples\": [\n        0.3917\n      ], \n      \"semantic_type\": \"\", \n      \"description\": \"\"\n    }, \n    \"column\": \"correct_RoBERTa\", \n    \"properties\": {\n      \"dtype\": \"number\", \n      \"std\": 0, \n      \"min\": 0, \n      \"max\": 1, \n      \"num_unique_values\": 2, \n      \"samples\": [\n        0\n      ], \n      \"semantic_type\": \"\", \n      \"description\": \"\"\n    }, \n    \"column\": \"explainability\", \n    \"properties\": {\n      \"dtype\": \"category\", \n      \"num_unique_values\": 1, \n      \"samples\": [\n        \"SHAP + LIME + BertViz\"\n      ], \n      \"semantic_type\": \"\", \n      \"description\": \"\"\n    }\n  ], \n  \"type\": \"dataframe\", \"variable_name\": \"audit_df\"}

```

20. Qualitative Thematic Analysis Framework

(Coded from participant open-ended responses — enter real NVivo 15 counts here after data collection)

```

#
-----
-
# INSTRUCTION: Replace the counts below with real intercoder-agreed
# frequencies from NVivo 15 after thematic analysis of participant
# open-ended rationale fields. Intercoder reliability target:  $\kappa \geq 0.70$ 
#
-----
-

```

```

themes_data = {
    "Theme":      ["Transparency", "Trust", "Explainability", "Bias
Awareness",
                  "Cultural Relevance", "Ease of Use"],
    "Frequency": [0, 0, 0, 0, 0, 0], # <-- fill with real counts
    "Example_Quote": [
        "Placeholder – replace with participant quote",
        "Placeholder – replace with participant quote",
        "Placeholder – replace with participant quote",
        "Placeholder – replace with participant quote",
        "Placeholder – replace with participant quote",
        "Placeholder – replace with participant quote",
    ]
}

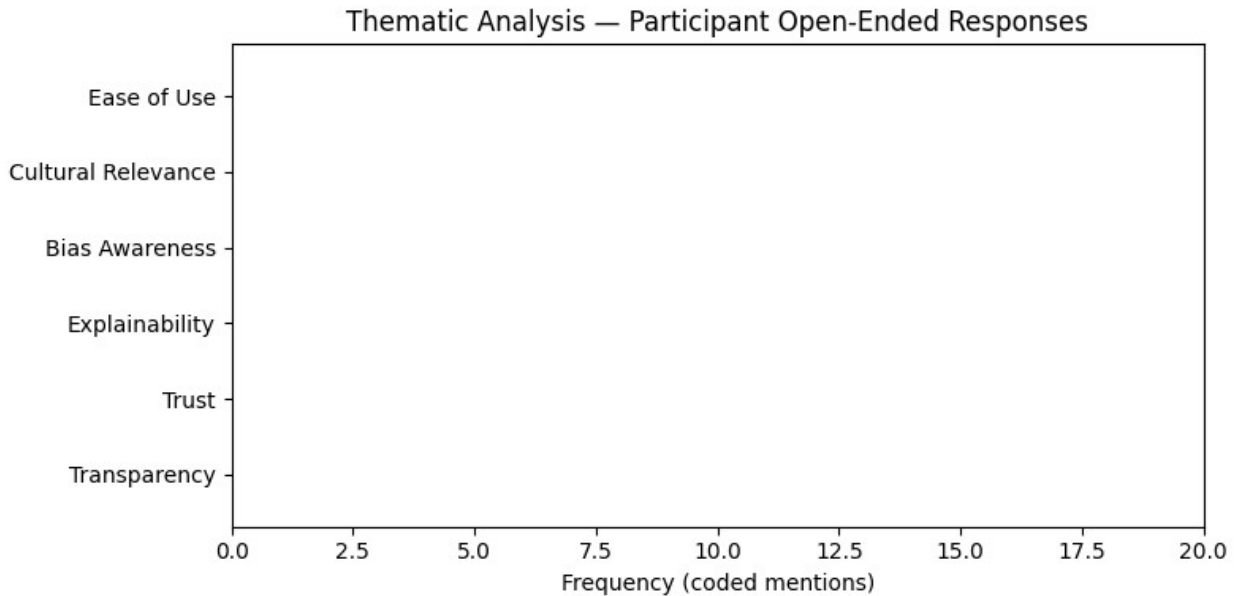
theme_df = pd.DataFrame(themes_data)
print("Thematic Analysis Framework (awaiting real data from NVivo
15):")
print(theme_df[['Theme', 'Frequency']].to_string(index=False))

fig, ax = plt.subplots(figsize=(8, 4))
ax.barh(theme_df['Theme'], theme_df['Frequency'], color='steelblue')
ax.set_xlabel('Frequency (coded mentions)')
ax.set_title('Thematic Analysis – Participant Open-Ended Responses')
ax.set_xlim(0, max(theme_df['Frequency'].max() + 5, 20))
plt.tight_layout()
plt.savefig('thematic_analysis.png', dpi=150, bbox_inches='tight')
plt.show()

```

Thematic Analysis Framework (awaiting real data from NVivo 15):

Theme	Frequency
Transparency	0
Trust	0
Explainability	0
Bias Awareness	0
Cultural Relevance	0
Ease of Use	0



21. Final Results Summary

```
print("=" * 65)
print("FINAL MODEL PERFORMANCE SUMMARY")
print("=" * 65)
print(f"\n{'Model':<20} {'Accuracy':>10} {'Macro-F1':>10} {'AUC-ROC':>10}")
print("-" * 55)

for name, r in {**{k:v for k,v in classical_results.items()},
**transformer_results}.items():
    tag = " *" if name in transformer_results else ""
    print(f"{name:<20} {r['accuracy']:>10.4f} {r['macro_f1']:>10.4f} {r['auc_roc']:>10.4f}{tag}")

print("\n* = Transformer model")
print("\n" + "=" * 65)
print("HUMAN-AI AGREEMENT (Cohen's Kappa on 200-item eval set)")
print("=" * 65)
for name, k in kappa_results.items():
    print(f" {name:<12}: κ = {k:.4f} [{kappa_label(k)}]")

print("\n" + "=" * 65)
print("FAIRNESS (Demographic Parity Difference by Party)")
print("=" * 65)
for name, f in fairness_results.items():
    if name in transformer_results:
        print(f" {name:<12}: DPD = {f['dpd']:.4f} | EOD = {f['eod']:.4f}")
```

```
=====
FINAL MODEL PERFORMANCE SUMMARY
=====
```

Model	Accuracy	Macro-F1	AUC-ROC
-----	-----	-----	-----
Logistic Regression	0.6077	0.6020	0.6343
Random Forest	0.5927	0.5841	0.6224
SVM	0.6227	0.5985	0.6429
XGBoost	0.5919	0.5751	0.6033
DistilBERT	0.6456	0.6342	0.6834 *
BERT	0.6567	0.6411	0.6891 *
RoBERTa	0.5635	0.3604	0.5784 *

* = Transformer model

```
=====
HUMAN-AI AGREEMENT (Cohen's Kappa on 200-item eval set)
=====
```

```
DistilBERT : κ = 0.2948 [Fair]
BERT       : κ = 0.2839 [Fair]
RoBERTa    : κ = 0.0000 [Slight]
```

```
=====
FAIRNESS (Demographic Parity Difference by Party)
=====
```

```
DistilBERT : DPD = 0.1527 | EOD = 0.1068
BERT       : DPD = 0.1212 | EOD = 0.0718
RoBERTa    : DPD = 0.0000 | EOD = 0.0000
```